

Universitatea Națională de Știință și Tehnologie POLITEHNICA
București

Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Segmentarea multi-label a structurilor fetale din ecografii abdominale

Proiect de diplomă

prezentat ca cerință parțială pentru obținerea titlului de
Inginer în domeniul *Calculatoare și Tehnologia Informației*
programul de studii de licență *Ingineria Informației (CTI – INF)*

Conducător științific

Prof. Dr. Ing. Corneliu Florea

Absolvent

Alex-Florin Vișoiu

Anul 2026

XXXX

add annex 1 xor 2 (formular de inscriere a temei lucrarii)

(from website with signatures + scan)

A handwritten signature in black ink, consisting of a stylized 'A' followed by a horizontal line and a small loop at the end.



Copyright © *anul 2026, Alex-Florin Vișoiu*

Toate drepturile rezervate

Autorul acordă UPB dreptul de a reproduce și de a distribui public copii pe hârtie sau electronice ale acestei lucrări, în formă integrală sau parțială.

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul “*Segmentarea multi-label a structurilor fetale din ecografii abdominale*”, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității Naționale de Știință și Tehnologie POLITEHNICA București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul *domeniul Calculatoare și Tehnologia Informației*, programul de studii *Ingineria Informației (CTI – INF)* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate.

Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare.

Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, *data*

Absolvent *Alex-Florin Vișoiu*



Cuprins

Introducere.....	19
Descrierea lucrării	19
Motivație.....	19
Obiective.....	20
Contribuții propuse.....	21
Structura lucrării	21
1 Context medical	23
1.1 Ecografia abdominală fetală.....	23
1.2 Anatomia structurilor de interes	24
1.2.1 Vena ombilicală	24
1.2.2 Artera aortică abdominală	24
1.2.3 Ficatul fetal.....	24
1.2.4 Stomacul fetal.....	24
1.3 Provocări specifice imaginilor ecografice	24
1.4 Necesitatea segmentării automate	25
2 Stadiul actual al cercetării.....	27
2.1 Segmentarea în imagistica medicală - abordări clasice și moderne	27
2.2 Arhitecturi de tip encoder-decoder pentru segmentare.....	27
2.3 Segmentarea pe imagini ecografice	28
2.4 Segmentarea structurilor fetale - particularități și lucrări existente.....	29
3 Fundamente teoretice ale rețelelor neuronale folosite.....	31
3.1 Rețele neuronale convoluționale (CNN).....	31
3.2 Arhitectura ResNet50 și conceptul de conexiuni reziduale	32
3.3 Arhitectura U-Net și segmentarea semantică	33
3.4 Transfer learning și inițializarea cu greutate pre-antrenate	34
3.5 Funcții de loss pentru segmentare (BCE, Dice, Tversky, Focal Tversky).....	36
3.6 Metrice de evaluare (Dice, IoU, sensibilitate, specificitate, precizie).....	37
4 Setul de date și preprocesarea	41
4.1 Descrierea setului de date	41
4.2 Formatul datelor și organizarea fișierelor	42
4.3 Împărțirea train/val/test la nivel de pacient	43
4.4 Augmentarea datelor (Albumentations).....	43
4.5 Adaptarea single-channel pentru intrarea ResNet50	45
5 Arhitectura propusă (MultiOutputUNet)	47
5.1 Schema generală a arhitecturii	47

5.2	Encoderul bazat pe ResNet50 modificat	48
5.3	Blocurile decoder cu skip connections	49
5.4	Stratul final și ieșirile multi-organ.....	51
5.5	Funcția de loss combinată	52
6	Procesul de antrenare.....	55
6.1	Configurația experimentală (hiperparametri, optimizator, scheduler)	55
6.2	Precizie mixtă și stabilizarea antrenării	55
6.3	Strategia de salvare a checkpoint-urilor.....	56
6.4	Mediul de antrenare	56
7	Experimente și rezultate	59
7.1	Experimentul 1 - Antrenare pe imagini brute (baseline)	59
7.2	Experimentul 2 - Antrenare pe imagini fără text suprapus	60
7.3	Experimentul 3 - Antrenare cu ponderi ajustate per organ	61
7.4	Experimentul 4 - Antrenare cu filtru CLAHE.....	62
7.5	Comparație globală între experimente	63
7.6	Rezultate la 90 de epoci	64
7.7	Rezultate la 150 de epoci	66
7.8	Analiză vizuală a predicțiilor	67
7.9	Concluzie generală a experimentelor.....	68
8	Aplicație demo pentru inferență interactivă	69
8.1	Prezentare generală.....	69
8.2	Arhitectura aplicației și tehnologii folosite	69
8.3	Interfața grafică	70
8.4	Pipeline-ul de preprocesare și inferență.....	72
8.5	Vizualizarea rezultatelor	73
8.6	Gestionarea erorilor și aspecte de implementare.....	75
	Concluzii și direcții viitoare	77
	Concluzii.....	77
	Contribuții personale	77
	Direcții viitoare	77
	Bibliografie	79
	Anexa 1	83
	Anexa 2	91

Listă de figuri

Figură 0.1 Ecografie abdominală cu organele evidențiate [23].....	20
Figură 1.1 Ecografie 2D - din baza de date [23].....	23
Figură 1.2 Ecografie 3D/4D [2].....	23
Figură 1.3 Exemplu de zgomot de tip speckle [23]	25
Figură 2.1 Model de rețea U-Net [9].....	28
Figură 2.2 Rezultatele obținute cu diferite arhitecturi de Liu et al. [13].....	30
Figură 3.1 Rețea neuronală complet-conectată (fully connected) [18].....	31
Figură 3.2 Rețea neuronală convoluțională (CNN) [19]	31
Figură 3.3 Encoder de tip ResNet50 [20].....	33
Figură 3.4 Arhitectura U-Net [21]	34
Figură 3.5 Analiza FROC pentru detecția polipilor, comparând șase variante de antrenare. [15]	35
Figură 4.1 Exemplu de imagine din setul de date [23]	41
Figură 4.2 P016_IMG7 [23].....	42
Figură 4.3 Structura fiecărui element din setul de date.....	42
Figură 4.4 Cele 4 măști din P01_IMG01 [23]	43
Figură 4.5 Toate augmentările, aplicate unei imagini [23]	44
Figură 5.1 Arhitectura modelului propus în lucrare.....	47
Figură 5.2 Structura unui bloc bottleneck din ResNet50. Adaptat după He et al. [10].	49
Figură 5.3 Structura unui bloc Decoder	50
Figură 5.4 Stratul final din rețea	52
Figură 7.1 Comparatie Ground-truth vs. Predicția modelului la 150 de epoci ale exp. 1.....	59
Figură 7.2 Comparatie Ground-truth vs. Predicția modelului la 150 de epoci ale exp. 2.....	60
Figură 7.3 Comparatie Ground-truth vs. Predicția modelului la 150 de epoci ale exp. 3.....	62
Figură 7.4 Comparatie Ground-truth vs. Predicția modelului la 150 de epoci ale exp. 4.....	63
Figură 7.5 Evoluția din timpul antrenării la 90 de epoci a mediei metricilor	64
Figură 7.6 Evoluția din timpul antrenării la 150 de epoci a mediei metricilor	66
Figură 8.1 Ecranul principal al aplicației	69
Figură 8.2 Aplicația după upload de imagine.....	71
Figură 8.3 Toolbar-ul / Legenda.....	71
Figură 8.4 Măștile de segmentare	72
Figură 8.5 Overlay-ul de organe pe imaginea originală.....	74
Figură 8.6 Overlay doar cu artera aortică.....	74

Listă de tabele

Tabel 6.1: Hiperparametrii folosiți pentru antrenare	55
Tabel 6.2: Detalii tehnice pentru antrenare	57
Tabel 7.1: Rezultatele la 90 de epoci pentru experimentul 1	64
Tabel 7.2: Rezultatele la 90 de epoci pentru experimentul 2	64
Tabel 7.3: Rezultatele la 90 de epoci pentru experimentul 3	65
Tabel 7.4: Rezultatele la 90 de epoci pentru experimentul 4	65
Tabel 7.5: Rezultatele la 150 de epoci pentru experimentul 1	66
Tabel 7.6: Rezultatele la 150 de epoci pentru experimentul 2	66
Tabel 7.7 Rezultatele la 150 de epoci pentru experimentul 3	67
Tabel 7.8: Rezultatele la 150 de epoci pentru experimentul 4	67

Listă de ecuații

Ecuția 3.1: Dice loss. [16].....	38
Ecuția 3.2: Tversky loss. [17]	39
Ecuția 3.3: Focal Tversky loss. [17].....	39
Ecuția 3.4 Formula Dice. [22].....	40
Ecuția 3.5 Formula IOU. [22]	40
Ecuția 3.6 Formula SENS. [22]	40
Ecuția 3.7 Formula <i>SPEC</i> . [22].....	40
Ecuția 3.8 Formula <i>PREC</i> . [22]	40
Ecuția 5.1 Formula Funcției de loss	55

Listă de acronime si abrevieri

1. Adam - Adaptive Moment Estimation (optimizator)
2. AMP - Automatic Mixed Precision - Antrenare cu precizie mixtă
3. BCE - Binary Cross-Entropy - Entropie încrucișată binară
4. BN - Batch Normalization - Normalizare pe loturi
5. BRATS - Brain Tumor Segmentation (deja în listă)
6. CLAHE - Contrast Limited Adaptive Histogram Equalization - Egalizare adaptivă a histogramei cu limitarea contrastului
7. CNN - Convolutional Neural Network (deja în listă)
8. COCO - Common Objects in Context (set de date)
9. CPU - Central Processing Unit - Unitate centrală de procesare
10. CT - Computed Tomography - Tomografie computerizată
11. CUDA - Compute Unified Device Architecture
12. DenseNet - Densely Connected Convolutional Network
13. DICE / DSC - Dice Similarity Coefficient - Coeficient de similaritate Dice
14. DICOM - Digital Imaging and Communications in Medicine
15. DOI - Digital Object Identifier
16. FN / FP / TN / TP - False Negative / False Positive / True Negative / True Positive - Fals negativ / Fals pozitiv / Adevărat negativ / Adevărat pozitiv
17. FROC - Free-response Receiver Operating Characteristic
18. GPU - Graphics Processing Unit - Unitate de procesare grafică
19. ILSVRC - ImageNet Large Scale Visual Recognition Challenge
20. IoU - Intersection over Union - Intersecție peste reuniune (indicele Jaccard)
21. ISBI - International Symposium on Biomedical Imaging
22. ISLES - Ischemic Stroke Lesion Segmentation (deja în listă)
23. LUNA16 - LUng Nodule Analysis 2016 (deja în listă)
24. mAP - mean Average Precision - Precizie medie
25. MHz - Megahertz
26. NPY - formatul binar de fișier NumPy
27. PNG - Portable Network Graphics
28. PREC - Precision - Precizie
29. R-CNN - Region-based Convolutional Neural Network
30. ReLU - Rectified Linear Unit - Unitate liniară rectificată
31. ResNet - Residual Network - Rețea reziduală
32. RGB / RGBA - Red, Green, Blue (, Alpha) - Roșu, Verde, Albastru (, Alfa)
33. RMN - Rezonanță Magnetică Nucleară

- 34. SENS - Sensitivity - Sensibilitate
- 35. SGD - Stochastic Gradient Descent - Coborâre stocastică pe gradient
- 36. SPEC - Specificity - Specificitate
- 37. U-Net - arhitectură (rețea în formă de U)
- 38. VGG / VGG-16 - Visual Geometry Group

Introducere

Descrierea lucrării

Lucrarea de față își propune rezolvarea problemei segmentarii multi-organ a 4 structuri anatomice fetale - vena ombilicală, artera aortică abdominală, ficatul și stomacul. Imaginile sunt obținute folosind ultrasunete în cadrul ecografiilor prenatale de rutină a femeilor în stadii avansate de sarcină. Este necesar un stadiu avansat al sarcinii pentru a se putea dezvolta și mai departe identifica aceste structuri anatomice.

Segmentarea unei imagini medicale reprezintă procesul de delimitare a unor regiuni specifice de interes la nivelul pixelilor, urmărindu-se identificarea și localizarea precisă a structurilor căutate. Imaginile medicale obținute cu ultrasunetele sunt cunoscute pentru greutatea cu care sunt interpretate, mai ales pentru persoane fără studii în domeniul medicinei. Soluția mea urmărește simplificarea procesului acesta.

Soluția propusă este bazată pe o arhitectură de rețea neuronală convoluțională U-Net, cu encoder de tip ResNet50 pre-antrenat pe imagini ImageNet, adaptat pentru imagini alb-negru, el fiind conceput pentru a segmenta imagini color (RGB). Rețeaua construită este antrenată să producă 4 măști de segmentare - câte una pentru fiecare structură fetală, tratând problema ca o sarcină de segmentare multi-label.

Lucrarea documentează un proces experimental incremental în care s-a plecat de la un model cât mai simplu, antrenat pe imaginile neprocesate, ajungând la o arhitectură adaptată la cerința problemei și la un tip de preprocesare care simplifică aspectul imaginilor, dar și ajută identificarea structurilor prin filtrarea imaginilor.

Motivație

Ecografia abdominală fetală reprezintă investigația pentru monitorizarea dezvoltării fătului în perioada prenatală. Procedura este neinvazivă, lipsită de riscuri precum radiații ionizante și accesibilă, ceea ce o face standardul în investigația morfologică, mai ales în cel de-al doilea trimestru al sarcinii.

Interpretarea este însă o sarcină complexă, dependentă în mare măsură de cel ce face interpretarea - de obicei medicul obstetrician. Calitatea imaginii variază în funcție de aparatura folosită, poziția fătului, compoziția corporală a mamei (dimensiunile uterului, strat adipos), iar structurile de interes pot fi dificil de delimitat vizual din cauza zgomotului specific acestui tip de imagini și contrastului scăzut.

Provocarea particulară a lucrării este reprezentată de diferența severă de scară între structurile segmentate. Ficatul fătului poate ocupa până la 40% din secțiunea prezentată în imagine, iar artera aortică abdominală poate fi de zeci de ori mai mică decât ficatul sau stomacul. (Fig 0.1). Aceste diferențe fac ca arhitectura clasică pentru segmentarea multi-label să obțină metrice mici precum 0.2 - 0.3 chiar și după 60 de epoci de antrenare.

Automatizarea segmentării poate reduce eroarea umană, dar mai important, poate ușura înțelegerea imaginilor de către pacient. De asemenea, poate accelera fluxul de lucru în cabinetul medicului obstetrician, dar cel mai important, poate oferi specialistului un suport în cazurile extrem de dificile, care ar necesita o a doua sau a treia părere de la alți medici.



Figură 0.1 Ecografie abdominală cu organele evidențiate [23]

Obiective

Lucrarea urmărește atingerea următoarelor obiective:

- Proiectarea și antrenarea unei arhitecturi pentru segmentarea multi-label a patru structuri anatomice.
- Adaptarea unui model pre-antrenat pe imagini color pentru imagini ecografice (în nuanțe de gri)
- Proiectarea unei funcții de loss și adaptarea ei astfel încât învățarea rețelei să fie cât mai eficientă
- Evaluarea progresului și evoluția modelului, fiecare etapă introducând o modificare dar și o schimbare în rezultate:
 - Imagini brute - baseline, exact cum sunt în baza de date
 - Eliminarea textului din marginile imaginilor
 - Schimbarea ponderilor fiecărui organ - weights
 - Aplicarea filtrului CLAHE
- Analiza calitativă a modelelor folosind metrici precum Dice, IoU, Precision, Recall, evaluate separat pentru fiecare structură fetală.
- Construirea unei interfețe simple ce va preprocesa, aplica modelul, post-procesa iar mai apoi afișa rezultatul obținut.

Contribuții propuse

Lucrarea propune o soluție de segmentare automată a patru structuri anatomice fetale din imagini ecografice abdominale, bazată pe o arhitectură de tip U-Net cu encoder ResNet50 pre-antrenat. Encoderul a fost adaptat pentru imagini grayscale prin modificarea primului strat convoluțional, iar rețeaua a fost antrenată să producă simultan patru măști de segmentare binare, câte una pentru fiecare structură.

O contribuție proprie a lucrării este pipeline-ul de preprocesare a imaginilor, care include un algoritm de eliminare a textului suprapus de echipamentul ecografic, implementat fără biblioteci specializate, și aplicarea filtrului CLAHE pentru îmbunătățirea contrastului local. Ambele etape au fost dezvoltate și ajustate incremental pe baza analizei rezultatelor obținute în fiecare experiment.

Configurația funcției de pierdere a fost adaptată problemei prin combinarea BCE ponderat per organ cu Focal Tversky loss, cu ponderi diferite pentru fiecare structură, pentru a compensa dezechilibrul sever de scară dintre artera aortică și celelalte organe.

În completarea modelului de segmentare, a fost dezvoltată o aplicație desktop de inferență interactivă, care permite aplicarea modelului pe imagini din dataset și vizualizarea rezultatelor prin overlay color cu opacitate controlabilă per organ.

Structura lucrării

- Lucrarea este organizată în opt capitole, fiecare acoperind o etapă distinctă a procesului de cercetare, de la contextul medical și teoretic până la experimentele proprii și aplicația demo.
- Capitolul 1 prezintă contextul medical al problemei. Sunt descrise ecografia abdominală fetală, anatomia celor patru structuri segmentate, provocările specifice imaginilor ecografice și necesitatea unui sistem de segmentare automată.
- Capitolul 2 trece în revistă stadiul actual al cercetării. Sunt prezentate abordările clasice și moderne din imagistica medicală, arhitecturile encoder-decoder utilizate în segmentare, particularitățile imaginilor ecografice și lucrările existente în domeniul segmentării structurilor fetale.
- Capitolul 3 acoperă fundamentele teoretice ale rețelelor neuronale folosite în soluția propusă: rețele convoluționale, arhitectura ResNet50 cu conexiunile sale reziduale, arhitectura U-Net, transfer learning, funcțiile de pierdere pentru segmentare (BCE, Dice, Tversky, Focal Tversky) și metricile de evaluare.
- Capitolul 4 descrie setul de date utilizat, formatul și organizarea fișierelor, împărțirea train/val/test la nivel de pacient, augmentările aplicate în antrenare și adaptarea encoderului ResNet50 pentru imagini grayscale.
- Capitolul 5 prezintă arhitectura propusă, numită MultiOutputUNet. Sunt descrise schema generală, encoderul bazat pe ResNet50 modificat, blocurile decoder cu skip connections, stratul final cu ieșiri per structură și funcția de pierdere combinată.
- Capitolul 6 detaliază procesul de antrenare: hiperparametrii și optimizatorul ales, antrenarea cu precizie mixtă și gradient clipping, strategia de salvare a checkpoint-urilor și mediul de calcul folosit.
- Capitolul 7 prezintă cele patru experimente realizate incremental, de la antrenarea pe imagini brute până la configurația finală cu filtru CLAHE și ponderi ajustate per organ. Fiecare experiment este evaluat cantitativ, cu tabele de metrici la 90 și 150 de epoci, și calitativ, prin analiza vizuală a predicțiilor.

- Capitolul 8 descrie aplicația desktop de inferență interactivă, implementată pentru evaluarea vizuală a modelului fără acces la mediul de antrenare.
- Lucrarea se încheie cu concluziile, contribuțiile personale și direcțiile de îmbunătățire identificate, urmate de bibliografie și anexe.

1 Context medical

1.1 Ecografia abdominală fetală

Ecografia este o metodă imagistică neinvazivă, bază de ultrasunete cu ajutorul căreia se pot investiga structurile corpului uman [1]. Reprezintă standardul pentru investigația medicală în sarcină, pentru că ofera medicului informații esențiale despre făt, placenta, cordon ombilical, lichid amniotic, col uterin, uter, ovare. Din acest motiv, în sarcină, ecografia este cea mai importantă investigație. De asemenea, ecografia este lipsită de riscuri, nu afectează fătul, și nu există limitare a numărului de examinări [1].

În lucrare, imaginile folosite pentru antrenare, testare și validare sunt ecografii fetale din trimestrul II. Se fac între săptămânile 20-22 și permit evaluarea dezvoltării fătului și a diverselor organe. Este considerată cea mai amănunțită ecografie, pentru că evaluează organele interne (ficat, stomac, rinichi, inima etc). Etichetele din baza de date folosită în prezenta lucrare reprezintă ficatul, stomacul, vena abdominală și artera aortică abdominală.

Această ecografie poate fi realizată 2D (Fig. 1.1), oferind cele mai multe informații despre anatomia fetală, 3D și 4D (Fig. 1.2), cele din urmă oferind imagini mult mai clare, dar neconcludente din punct de vedere medical [1].



Figură 1.1 Ecografie 2D - din baza de date [23]



Figură 1.2 Ecografie 3D/4D [2]

1.2 Anatomia structurilor de interes

1.2.1 Vena ombilicală

Vena ombilicală reprezintă o parte importantă a sistemului circulator fetal. Spre deosebire de venele obișnuite ale corpului adult, vena ombilicală transportă sânge oxigenat de la placentă spre fătul în creștere [3]. În timpul dezvoltării fătului, vena ombilicală pornește de la placentă, trece prin cordonul ombilical alături de artera aortică abdominală, ajungând la făt, pentru a-l alimenta cu oxigen de la mamă. La ecografia morfologică, identificarea eventualelor anomalii ale acesteia este esențială pentru monitorizarea dezvoltării sanatoase a fătului și în prevenirea complicațiilor prenatale și postnatale [3].

1.2.2 Artera aortică abdominală

Aorta abdominală este artera principală a cavității abdominale. Pornește de la nivelul vertebrei T12, acolo unde aorta toracică traversează diafragma, și coboară de-a lungul coloanei până la L4, unde se divide în arterele iliace comune [4]. Pe traseu vascularizează organele abdominale prin mai multe ramuri: artera celiacă, arterele renale și arterele mezenterice. În ecografia fetală apare ca o structură mică, rotundă și pulsatilă, plasată posterior în abdomen, lângă coloana vertebrală [4].

1.2.3 Ficatul fetal

Ficatul este unul dintre primele organe vizibile în ecografiile fetale. Se dezvoltă încă din săptămâna a patra a sarcinii [5]. Rolurile îndeplinite de acesta în dezvoltarea fătului, dar și în pregătirea nașterii sunt: metabolismul, digestia și homeostaza. Ocupă un volum semnificativ din cavitatea abdominală a fătului, fiind esențial dezvoltării acestuia. Aceste caracteristici structurale și funcționale fac din ficatul fetal un organ de referință în ecografia obstetricală, dar și o țintă relevantă pentru segmentarea automată. [5]

1.2.4 Stomacul fetal

Stomacul fetal este singura structură a tractului digestiv vizibilă în mod normal la ecografia morfologică din trimestrul al doilea. [30] Procesul de înghițire apare în săptămâna 11 de sarcină, pentru a pregăti tractul digestiv. Dimensiunea sa variază considerabil de la un făt la altul, lucru ce poate fi observat și în baza de date folosită pentru antrenarea modelului din lucrare. Absența stomacului, sau dimensiunile lui foarte mici, pot semnala o formă de atrezie esofagiană, în timp ce dimensiunile foarte mari pot semnala obstrucție distală. Vizualizarea stomacului, și segmentarea acestuia contribuie enorm la prevenirea problemelor și afecțiunilor asociate cu acesta, încă din faza prenatală. [30]

1.3 Provocări specifice imaginilor ecografice

Imaginile ecografice prezintă o serie de caracteristici care le diferențiază fundamental de alte modalități imagistice medicale, precum CT sau RMN, și care îngreunează considerabil procesul de segmentare automată.

Cea mai importantă provocare este zgomotul de tip speckle [6], specific oricărei imagistici coerente, inclusiv ultrasunetelor. Acesta apare din cauza sumării ecourilor reflectate de țesuturi la

scară microscopică [6]. Textura granulară rezultată nu corespunde structurii anatomice reale a țesutului, ci este un artefact al procesului de achiziție. Distribuția statistică a amplitudinii semnalului urmează o distribuție Rayleigh în condiții de zgomot speckle complet dezvoltat, ceea ce face ca metodele clasice de procesare a imaginilor să nu fie direct aplicabile. [6]

Un alt impediment îl reprezintă contrastul scăzut dintre structurile de interes și țesuturile adiacente. Spre deosebire de CT, unde diferențele de densitate produc contraste clare, în ecografie regiunile anatomice învecinate pot avea valori de intensitate similare, făcând delimitarea contururilor dificilă. [6]



Figură 1.3 Exemplu de zgomot de tip speckle [23]

Orientarea transductorului față de suprafața structurii examinate influențează direct calitatea conturului în imagine. [6] Când unghiul de incidență nu este perpendicular pe suprafața structurii, ecoul returnat este slab sau absent, rezultând contururi incomplete în anumite zone. Acest fenomen, numit signal dropout, afectează în mod special structurile de dimensiuni mici, cum este artera aortică abdominală.

Atenuarea semnalului cu adâncimea reprezintă o altă sursă de variabilitate. Undele ultrasonice își pierd din energie pe măsură ce traversează țesuturile, iar structurile situate mai adânc apar cu un contrast mai scăzut. În ecografiile prenatale, acest efect poate fi accentuat de grosimea stratului adipos al mamei și de poziția fătului. [6]

1.4 Necesitatea segmentării automate

Ecografia este una dintre cele mai folosite metode de investigație medicală din lume, cu sute de milioane de examinări efectuate în fiecare an. În obstetrică, ecografia abdominală fetală face parte din monitorizarea standard a sarcinii, ceea ce înseamnă că numărul de imagini care ajung să fie interpretate zilnic este foarte mare. [8]

Interpretarea rămâne un proces manual. Medicul se uită la imagine, identifică structurile și le delimitează pe baza experienței proprii. Problema e că doi medici pot ajunge la rezultate diferite pe aceeași imagine [7], mai ales când structurile sunt mici sau au un contrast slab față de țesuturile din jur. Această variabilitate nu e neglijabilă și poate influența decizia clinică.

Un sistem de segmentare automată nu rezolvă toate aceste probleme, dar poate reduce dependența de subiectivitatea operatorului. În cazurile clare, poate accelera fluxul de lucru. În cazurile dificile, poate oferi un punct de referință suplimentar, fără să înlocuiască judecata medicului.

Un alt aspect, mai puțin discutat, este înțelegerea imaginilor de către pacient. O ecografie în nuanțe de gri e greu de interpretat pentru cineva fără pregătire medicală. Structurile evidențiate prin măști de segmentare colorate fac rezultatul mai vizibil și mai ușor de explicat în cabinet. [7]

2 Stadiul actual al cercetării

2.1 Segmentarea în imagistica medicală - abordări clasice și moderne

Segmentarea imaginilor medicale are o istorie îndelungată, iar metodele folosite de-a lungul timpului reflectă evoluția generală a procesării imaginilor și a inteligenței artificiale.

Primele abordări, dezvoltate începând cu anii '70, se bazau pe procesarea la nivel de pixel: filtre de detecție a marginilor, algoritmi de region-growing și modele matematice simple [6]. Sistemele construite astfel erau rigide și greu de generalizat, funcționând bine doar în condiții foarte specifice.

Spre sfârșitul anilor '90, atenția s-a mutat spre metode supervizate, bazate pe extragerea manuală de trăsături și clasificatori statistici. Modele active-shape și metodele bazate pe atlas au devenit abordările standard pentru segmentarea structurilor anatomice. Aceste metode au dat rezultate bune în condiții controlate, dar rămăneau dependente de calitatea trăsăturilor extrase manual de cercetători. [6]

Pe imagini ecografice, limitările acestor metode sunt și mai evidente. Metodele bazate pe gradient de intensitate, spre exemplu, funcționează bine pe imagini CT sau RMN, unde marginile structurilor sunt clare. Pe ecografii, speckle-ul generează răspunsuri false de gradient, atenuarea semnalului cu adâncimea slăbește marginile structurilor profunde, iar anizotropia achiziției poate duce la contururi lipsă. Autorii Noble și Boukerroui argumentează că, spre deosebire de CT și RMN, ecografia necesită metode adaptate care să modeleze fizica achiziției. [6]

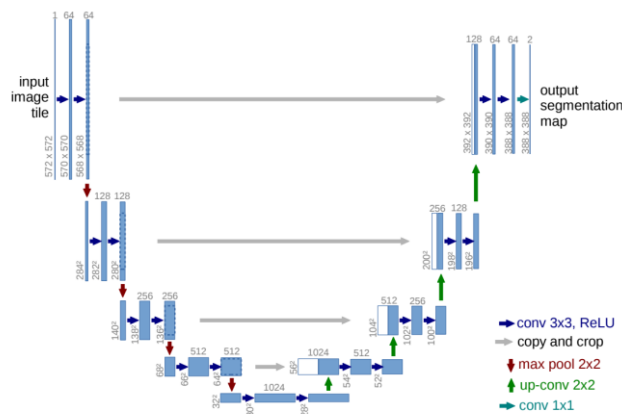
Momentul de cotitură a venit în 2012, odată cu succesul rețelei AlexNet la competiția ImageNet. CNN-urile au demonstrat că pot învăța automat trăsături relevante direct din date, eliminând necesitatea extragerii manuale [7]. Tranziția în imagistica medicală a fost graduală, dar rapidă, iar dintr-un studiu din 2017 care analizează peste 300 de lucrări reiese că CNN-urile antrenate au devenit abordarea dominantă, înlocuind metodele clasice în majoritatea aplicațiilor.

La competițiile majore de segmentare medicală, BRATS pentru tumori cerebrale, ISLES pentru accidente vasculare, LUNA16 pentru noduli pulmonari, echipele de top au folosit în toate cazurile arhitecturi bazate pe CNN-uri. Un aspect important subliniat în literatură este că arhitectura exactă a rețelei nu este factorul decisiv, ci tehnicile de preprocesare și augmentare a datelor fac adesea diferența între rezultate slabe și rezultate bune. [7]

2.2 Arhitecturi de tip encoder-decoder pentru segmentare

Arhitecturile de tip encoder-decoder sunt în prezent abordarea standard pentru segmentarea semantică în imagistica medicală. Procesul e împărțit în două etape: o cale de contracție, care extrage trăsături din imagine la rezoluții din ce în ce mai mici, și o cale de expansiune, care reconstruiește harta de segmentare la rezoluția originală.

U-Net, propus de Ronneberger et al. [9] în 2015, a devenit arhitectura de referință în domeniu. Calea de contracție funcționează ca un encoder clasic, captând contextul imaginii prin convoluții și operații de pooling. Calea de expansiune înlocuiește operațiile de pooling cu upsampling, măbind progresiv rezoluția ieșirii. O particularitate importantă a arhitecturii este absența straturilor fully connected, rețeaua operând exclusiv cu convoluții. [9]



Figură 2.1 Model de rețea U-Net [9]

Elementul cheie al U-Net sunt skip connections, conexiuni directe între straturile corespondente din encoder și decoder. Prin acestea, trăsăturile de înaltă rezoluție din calea de contracție sunt concatenate cu ieșirea pe care s-a făcut upsampling din calea de expansiune, permițând rețelei să combine informația de context cu informația spațială precisă. Fără aceste conexiuni, detaliile fine ale contururilor s-ar pierde în procesul de downsampling. [9]

Pentru encoderul arhitecturii propuse în lucrarea de față s-a folosit ResNet50, propus de He et al. [10] în 2015. Motivul pentru care ResNet a reprezentat un pas important ține de o problemă concretă: pe măsură ce rețelele devin mai adânci, eroarea de antrenare crește în loc să scadă, fenomen care nu se explică prin overfitting. [10]

Blocul rezidual rezolvă problema prin învățarea rezidului față de identitate. În loc să învețe direct transformarea $H(x)$, rețeaua învață $F(x) = H(x) - x$, iar ieșirea finală e $F(x) + x$. Straturile convoluționale învață doar ce trebuie adăugat peste intrare, nu întreaga transformare de la zero. Această idee simplă, implementată prin shortcut connections care adună intrarea direct peste ieșirea blocului, face antrenarea rețelelor foarte adânci stabilă și eficientă, fără a adăuga parametri sau complexitate computațională suplimentară. [10]

2.3 Segmentarea pe imagini ecografice

Imaginile ecografice prezintă o serie de particularități care le fac mai dificil de procesat față de alte modalități imagistice medicale, cum ar fi CT sau RMN. Imaginile ecografice au un raport semnal-zgomot scăzut și sunt afectate frecvent de artefacte de tip acoustic shadowing, mai ales în apropierea structurilor osoase. Contrastul dintre structura de interes și țesuturile din jur este de multe ori insuficient pentru o delimitare clară. La acestea se adaugă variabilitatea introdusă de echipament și de operator, care face ca imaginile să difere semnificativ de la o examinare la alta.

Baumgartner et al. [11] arată că această variabilitate are un impact direct asupra modelelor antrenate: un model care funcționează bine pe imagini de la un anumit echipament sau operator nu se transferă neapărat la alt context de achiziție. Autorii raportează că ghidarea transducerului la planul corect de scanare, prin anatomia înalt variabilă a pacientului, necesită ani de pregătire și produce reproductibilitate scăzută chiar și în rândul specialiștilor experimentați. [11]

Pe lângă aceste aspecte tehnice, imaginile ecografice fetale introduc un nivel suplimentar de complexitate. Poziția fătului poate face imposibilă obținerea unui plan de scanare clar pentru anumite structuri. Burgos-Artizzu et al. [12] menționează explicit că variabilitatea intra-clasă ridicată și dezechilibrul compozițional al seturilor de date reflectă direct condițiile dintr-un scenariu clinic real, nu limitări ale procesului de colectare. [12]

Utilizarea rețelelor convoluționale pe imagini ecografice rămâne mai puțin studiată față de alte modalități imagistice. Totuși, rezultatele recente arată că arhitecturile CNN pot atinge performanțe comparabile cu cele ale specialiștilor umani pe anumite sarcini. SonoNet, arhitectura propusă de Baumgartner et al., bazată pe VGG16 și adaptată pentru detecție în timp real, obține un F1-score mediu de 0,798 pe detecția planurilor standard fetale dintr-un flux video continuu. Burgos-Artizzu et al. raportează o acuratețe medie de 93,6% pentru clasificarea planurilor comune, obținută cu DenseNet-169, performanță similară cu cea a tehnicienilor evaluați în același studiu. [11] [12]

Aceste rezultate sunt obținute însă pe sarcini de clasificare sau detecție, nu de segmentare propriu-zisă. Segmentarea pixel cu pixel pe imagini ecografice rămâne o problemă mai dificilă, mai ales când structurile de interes sunt mici, cu contur slab definit sau parțial ascunse de artefacte. Acesta este și cazul lucrării de față, unde structuri cu dimensiuni foarte diferite, cum ar fi artera aortică abdominală și ficatul fetal, trebuie segmentate simultan din aceeași imagine.

2.4 Segmentarea structurilor fetale - particularități și lucrări existente

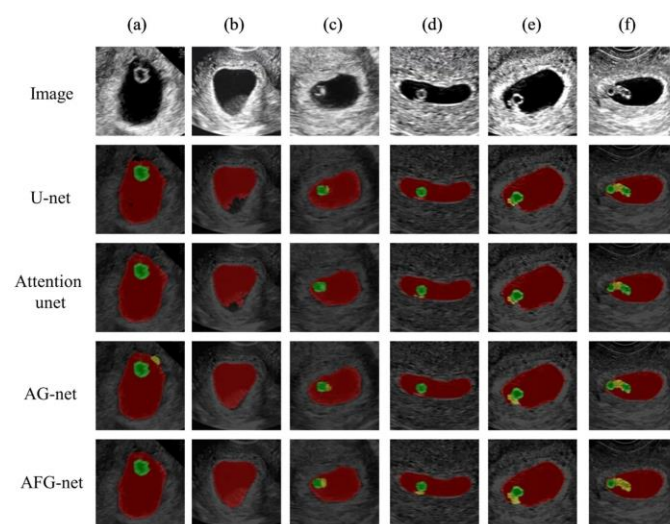
Segmentarea automată a structurilor fetale din imagini ecografice este un domeniu relativ restrâns în literatura de specialitate. Majoritatea lucrărilor existente se concentrează pe măsurători biometrice standard - circumferința craniană, circumferința abdominală, lungimea femurului - și nu pe segmentarea structurilor interne. Lucrări care abordează explicit segmentarea venei sau arterei aortice abdominale în imagini 2D nu au fost identificate în literatura consultată.

Metodele clasice de segmentare, precum region growing, active contours sau threshold segmentation, s-au dovedit fragile pe imagini ecografice, din cauza zgomotului ridicat și a contrastului scăzut. Abordările bazate pe rețele convoluționale au adus îmbunătățiri clare, U-Net devenind arhitectura de referință pentru segmentarea în imagistica medicală.

Liu et al. [13] propun AFG-net, o variantă modificată de U-Net cu module de attention fusion și guide filter, aplicată pe segmentarea sacului gestațional, sacului vitelin și embrionului în ecografii transvaginale din trimestrul I. Rețeaua obține un Dice mediu de 92,11% pe cele trei structuri, față de 89,67% pentru U-Net standard. Autorii observă că operațiile repetate de down-sampling reduc rezoluția și afectează segmentarea structurilor mici - embrionul, cea mai mică structură din set, înregistrează cel mai mic Dice, 86,11%. [13]

Problema diferenței de scară apare și în lucrarea de față, dar mai accentuat. Ficatul fetal ocupă o parte semnificativă din imagine, în timp ce artera aortică abdominală poate fi de zeci de ori mai mică. O arhitectură care nu tratează acest dezechilibru va performa bine pe structurile mari și slab pe cele mici.

Liu et al. [13] menționează și variabilitatea inter-observator ca limitare: sonografi diferiți pot produce contururi diferite pentru aceleași structuri, ceea ce introduce incertitudine în ground truth-ul folosit la antrenare. Această problemă este comună în domeniu și afectează direct interpretarea metricilor raportate. [13]

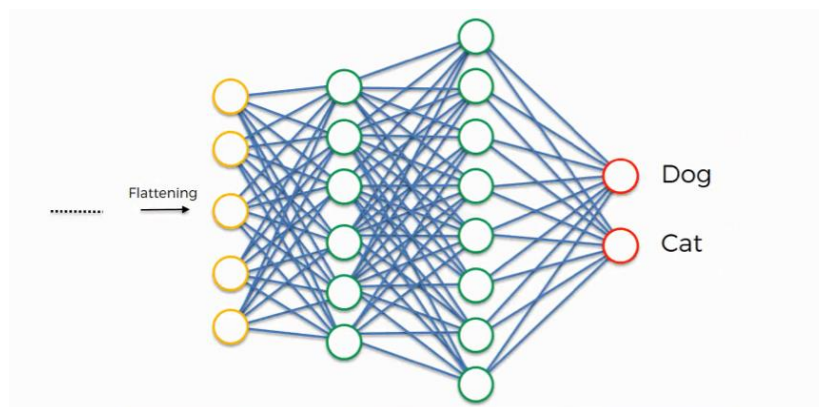


Figură 2.2 Rezultatele obținute cu diferite arhitecturi de Liu et al. [13]

3 Fundamente teoretice ale rețelelor neuronale folosite

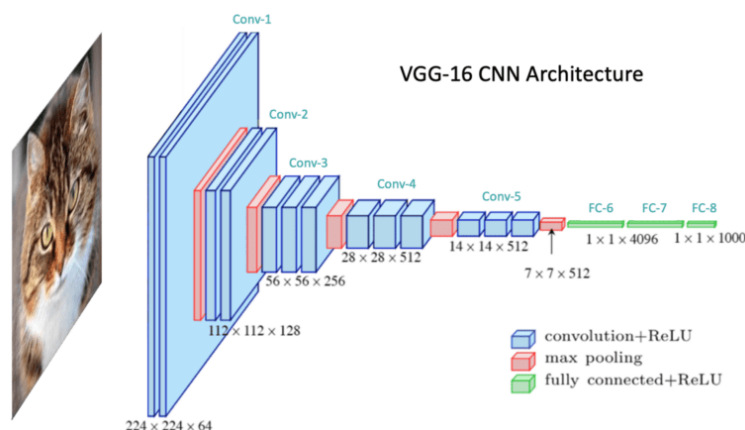
3.1 Rețele neuronale convoluționale (CNN)

O rețea neuronală convoluțională este construită în jurul ideii că pixelii dintr-o imagine nu sunt independenți [14], cei vecini sunt corelați între ei. Rețelele fully-connected clasice ignoră această structură spațială, conectând fiecare neuron la toți neuronii din stratul anterior, după cum se observa din figura 3.1 atașată mai jos. [14]



Figură 3.1 Rețea neuronală complet-conectată (fully connected) [18]

O rețea convoluțională exploatează în schimb localitatea: fiecare filtru examinează o regiune mică din imagine, nu întreaga intrare [14]. Componenta centrală care face asta posibil este stratul convoluțional. Acesta aplică un set de filtre numite și kerneluri asupra imaginii de intrare. Fiecare filtru alunecă peste imagine și produce o hartă de activare (feature map), care codifică prezența unui anumit pattern local în fiecare poziție spațială. Filtrele din straturile de început detectează de regulă caracteristici simple muchii, contraste iar cele din straturile mai adânci combină aceste caracteristici în reprezentări din ce în ce mai abstracte. [14]



Figură 3.2 Rețea neuronală convoluțională (CNN) [19]

După fiecare strat convoluțional se aplică de obicei o funcție de activare. Funcțiile clasice de tip sigmoid sau tanh au dezavantajul că gradientul lor devine neglijabil pentru valori mari ale

intrării, ceea ce încetinește antrenarea. ReLU definit ca $f(x) = \max(0, x)$ evită acest fenomen, deoarece nu saturează pentru valori pozitive. Krizhevsky et al. [14] au demonstrat că o rețea antrenată cu ReLU ajunge la același nivel de eroare de aproximativ șase ori mai rapid decât una cu activări tanh, ceea ce a făcut ReLU standardul de facto în arhitecturile convoluționale moderne. [14]

Un alt element frecvent întâlnit este stratul de pooling. Acesta reduce dimensiunea spațială a hărților de activare rezumând un grup de valori vecine printr-o singură valoare de obicei maximul (max pooling). Efectul este dublu: scade costul computațional al straturilor următoare și introduce un grad de invarianță la translații mici ale obiectului în imagine. [14]

Regularizarea rețelei în timpul antrenării se face prin mai multe metode. Una dintre cele documentate în [14] este dropout: la fiecare pas de antrenare, fiecare neuron dintr-un strat ascuns are o probabilitate de 0.5 de a fi dezactivat temporar. Neuronul nu contribuie nici la propagarea înainte, nici la backpropagation în acel pas. Efectul practic este că rețeaua nu poate conta pe prezența unor neuroni specifici și este forțată să distribuie informația mai uniform, reducând supraantrenarea (overfitting). [14]

Importanța acestor arhitecturi a fost demonstrată clar în 2012, când Krizhevsky et al. [14] au obținut o rată de eroare top-5 de 15.3% la competiția ILSVRC, față de 26.2% a celui de-al doilea clasat o diferență de peste 10 puncte procentuale față de metodele clasice. Acest rezultat a marcat momentul în care rețelele convoluționale adânci au devenit direcția principală în viziunea artificială. [14]

3.2 Arhitectura ResNet50 și conceptul de conexiuni reziduale

Pe măsură ce rețelele convoluționale devin mai adânci, apare un fenomen contraintuitiv: acuratețea pe setul de antrenament nu crește, ci se degradează. He et al. [10] arată explicit că o rețea de 56 de straturi are o eroare de antrenare mai mare decât una de 20 de straturi pe tot parcursul antrenamentului. Nu este vorba de supraantrenare, deoarece problema apare pe setul de antrenament, nu pe cel de test. Cauza este una de optimizare: optimizatorii clasici de tip SGD nu reușesc să găsească soluția corectă pentru rețele foarte adânci în timp rezonabil. [10]

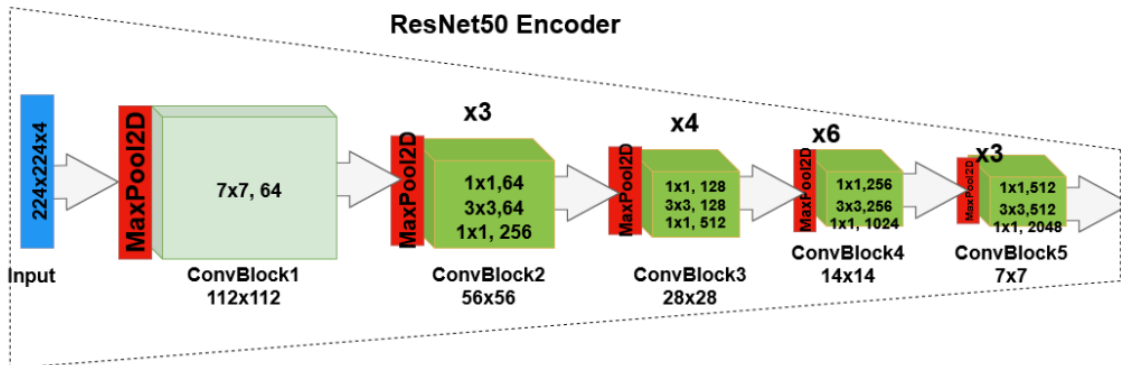
Soluția propusă de autori este residual learning. Fie $H(x)$, funcția pe care un bloc de straturi trebuie să o aproximeze, unde x este intrarea în bloc. În loc să lase straturile să învețe direct $H(x)$, autorii le cer să învețe reziduul $F(x) = H(x) - x$. Funcția originală devine astfel $F(x) + x$. Dacă soluția optimă pentru un bloc este aproape de identity mapping, adică straturile respective nu ar trebui să modifice semnificativ intrarea, atunci este mult mai ușor pentru optimizator să ducă $F(x)$ spre zero decât să reconstituie explicit $H(x) = x$ printr-un cumul de neliniarități. [10]

Acest lucru se implementează printr-o conexiune shortcut: intrarea x este adunată la nivel de element cu ieșirea blocului, fără a trece prin niciun strat cu parametri. Conexiunea nu adaugă niciun parametru și nicio complexitate computațională suplimentară. [10]

Pentru rețele adânci de tipul ResNet50, blocul de bază cu două straturi convoluționale 3×3 devine costisitor computațional. Se folosește în schimb un bloc bottleneck format din trei straturi: o convoluție 1×1 care reduce dimensionalitatea, o convoluție 3×3 care operează la dimensionalitatea redusă și o convoluție 1×1 care restaurează dimensionalitatea inițială. Cele două blocuri au un timp de calcul similar, dar bottleneck-ul permite adăugarea unui strat 3×3 la un cost redus. [10]

ResNet50 este organizat în cinci stagii. Primul strat este o convoluție 7×7 cu 64 de filtre și stride 2, urmată de max pooling. Urmează patru grupuri de blocuri bottleneck: conv2_x cu 3 blocuri la dimensiune 56×56 , conv3_x cu 4 blocuri la 28×28 , conv4_x cu 6 blocuri la 14×14 și conv5_x cu 3

blocuri la 7x7. La final se aplică average pooling global și un strat fully-connected cu softmax. Numărul total de straturi ponderate este 50. [10]



Figură 3.3 Encoder de tip ResNet50 [20]

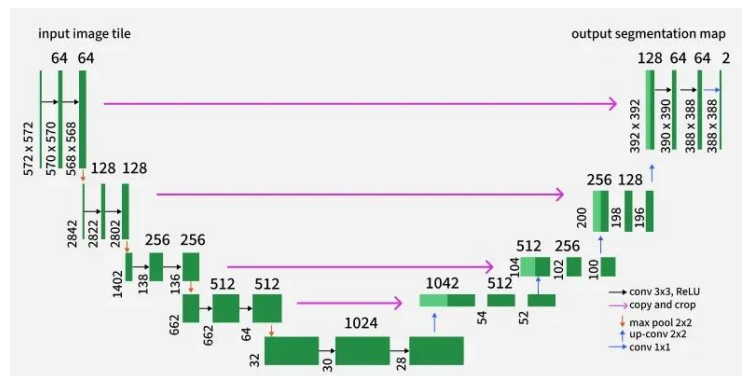
Pe ImageNet, ResNet50 obține o eroare top-5 de 6.71% la testare cu 10 crop-uri, față de 9.33% pentru VGG-16. He et al. [10] arată că înlocuirea unui backbone VGG-16 cu ResNet-101 în Faster R-CNN crește mAP pe COCO de la 21.2% la 27.2%, diferența provenind exclusiv din calitatea reprezentărilor extrase. Acesta este argumentul direct pentru utilizarea ResNet50 ca encoder prețrenat într-o arhitectură de segmentare: feature map-urile produse de nivelele intermediare sunt suficient de bogate semantic pentru a nu fi necesară antrenarea de la zero. [10]

3.3 Arhitectura U-Net și segmentarea semantică

Rețelele convoluționale clasice din 2015 erau concepute pentru clasificare la nivel de imagine: o imagine intră, o etichetă iese. Segmentarea semantică cere ceva fundamental diferit: fiecărui pixel din imagine trebuie să i se atribuie o etichetă de clasă. Output-ul nu mai este un vector de probabilități, ci o hartă de dimensiunile imaginii de intrare.

Abordarea anterioară, cu straturi complet-conectate, rezolva această problemă prin inferență patch cu patch: pentru fiecare pixel, rețeaua primea o regiune locală centrată pe el și prezicea clasa acelui pixel. Problema era dublă. Pe de o parte, viteza era extrem de redusă, deoarece rețeaua trebuia rulată separat pentru fiecare pixel, cu o redundanță enormă între patch-urile vecine. Pe de altă parte, exista un compromis inevitabil între context și localizare: patch-uri mari ofereau mai mult context global, dar mai mult pooling însemna pierdere de rezoluție și localizare mai slabă; patch-uri mici păstrau detaliile spațiale, dar contextul era insuficient. Ronneberger et al. [9] pornesc explicit de la aceste două limitări.

U-Net este organizat în forma unui U simetric și conține două căi distincte. Călea contractantă, sau encoderul, captează contextul semantic: rezoluția spațială scade progresiv prin max pooling, în timp ce numărul de canale crește. La capătul acestei căi, feature maps au rezoluție minimă și conțin informație semantică bogată despre ce este prezent în imagine, dar au pierdut informația despre unde anume se află structurile. Călea expansivă, sau decoderul, reconstruiește rezoluția spațială și produce harta de segmentare la dimensiunea imaginii originale. Upsampling-ul simplu pe o reprezentare comprimată ar produce contururi neclare; de aceea, decoderul primește informație suplimentară din encoder prin skip connections. [9]



Figură 3.4 Arhitectura U-Net [21]

Fiecare nivel al encoderului conține două convoluții 3x3 consecutive, fiecare urmată de ReLU, și un max pooling 2x2 cu stride 2. La fiecare pas de downsampling, numărul de canale se dublează, de la 64 la 128, 256, 512, ajungând la 1024 în bottleneck. Această regulă de design compensează pierderea de informație spațială prin bogăție crescută pe axa canalelor. [9]

Fiecare nivel al decoderului constă dintr-o up-convoluție 2x2 care dublează rezoluția și înjumătățește numărul de canale, urmată de concatenarea cu feature maps corespunzătoare din encoder, după care se aplică din nou două convoluții 3x3 cu ReLU. Concatenarea combină semantica bogată a decoderului cu detaliile spațiale fine ale encoderului, rezolvând tocmai trade-off-ul din abordarea prin patch-uri. [9]

Skip connections din U-Net sunt diferite față de cele din ResNet. În ResNet, shortcut-ul adună element-wise feature maps din același bloc pentru a facilita optimizarea. În U-Net, conexiunile leagă niveluri diferite de rezoluție de pe cele două căi și combină prin concatenare, nu prin adunare, dublând numărul de canale la intrarea în fiecare bloc al decoderului. [9]

Stratul final aplică o convoluție 1x1 care mapează vectorul de 64 de canale al fiecărui pixel la numărul de clase dorit, urmată de softmax pixel-wise. Rețeaua nu conține niciun strat fully-connected, ceea ce îi permite să proceseze imagini de orice dimensiune. [9]

O proprietate importantă a arhitecturii este că funcționează cu date puține, o cerință esențială în imagistica medicală unde adnotarea cere experți și este costisitoare. Ronneberger et al. [9] compensează lipsa datelor prin augmentare cu deformări elastice aleatoare, care permit rețelei să învețe invarianța la variațiile naturale ale țesuturilor fără a le vedea explicit în datele adnotate.

Rezultatele raportate confirmă eficiența arhitecturii. Pe ISBI Cell Tracking Challenge 2015, U-Net obține un IOU de 0.9203 pe setul PhC-U373, față de 0.83 al celei de-a doua metode clasate, și un IOU de 0.7756 pe setul DIC-HeLa, față de 0.46. Pe EM segmentation challenge, obține cel mai bun warping error fără niciun post-procesare suplimentară, depășind metoda Ciresan et al. bazată pe sliding window. [9]

3.4 Transfer learning și inițializarea cu greutatea pre-antrenate

Antrenarea unui CNN adânc de la zero necesită cantități mari de date etichetate. În imagistica medicală, această cerință este greu de satisfăcut: adnotarea imaginilor necesită experți, este costisitoare și lentă, iar structurile sau leziunile de interes sunt adesea rare în dataset-uri. Tajbakhsh et al. [15] identifică trei obstacole principale pentru antrenarea de la zero în domeniul medical: lipsa datelor adnotate, resursele computaționale ridicate și dificultățile de convergență, care necesită ajustări repetitive ale arhitecturii și hiperparametrilor.

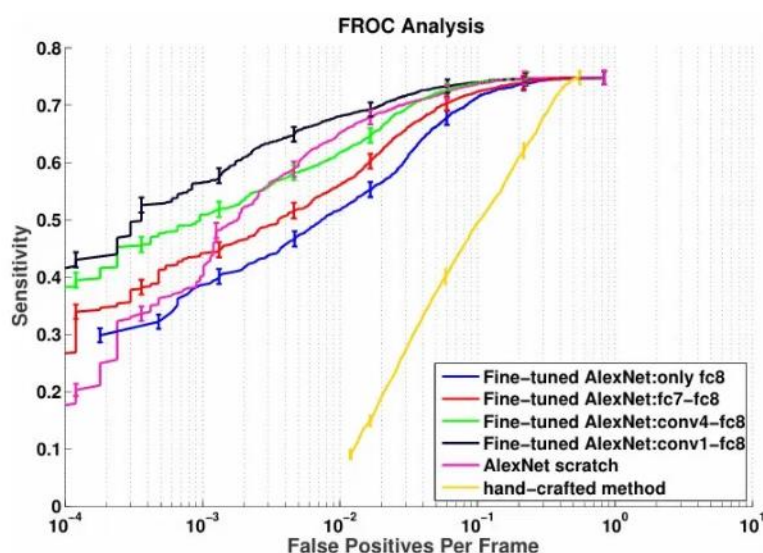
Transfer learning rezolvă parțial aceste probleme. În loc să pornească de la greutatea inițializată aleator, rețeaua preia greutatea unui model pre-antrenat pe un dataset mare, de obicei ImageNet, și le folosește ca punct de plecare. Straturile timpurii ale unui CNN detectează caracteristici de nivel scăzut: muchii, texturi, gradienti. Aceste reprezentări sunt utile indiferent de domeniu, inclusiv în imagistica medicală. Straturile profunde conțin reprezentări de nivel înalt, specifice aplicației originale, și acestea sunt cele care se re-antrenează pe datele medicale. [15]

Procesul de adaptare se numește fine-tuning. Concret, greutatea rețelei pre-antrenate sunt copiate în rețeaua țintă, cu excepția ultimului strat fully-connected, care se înlocuiește cu unul nou dimensionat pentru numărul de clase al aplicației medicale. Rata de învățare per strat controlează ce se actualizează: straturile cu rată zero rămân fixe, celelalte se adaptează la datele noi. [15]

O întrebare practică este câte straturi trebuie deblocate. Tajbakhsh et al. [15] testează sistematic opt variante, de la deblocarea doar a ultimului strat până la deblocarea completă a rețelei. Concluzia este că nu există o adâncime universală optimă: depinde de cât de diferite sunt imaginile medicale față de cele naturale din ImageNet. Pentru modalități mai apropiate de imaginile naturale, fine-tuning moderat este suficient. Pentru ecografii sau CT, unde diferența față de ImageNet este mai mare, este nevoie de deblocarea și a straturilor convoluționale timpurii.

Tajbakhsh et al. [15] rezolvă incompatibilitatea dintre arhitectura ResNet50 și imaginile medicale grayscale printr-o adaptare simplă: canalul gri este repetat de trei ori, generând un input de trei canale compatibil cu intrarea rețelei pre-antrenate. Autorii confirmă că metoda funcționează în practică pe ecografii și imagini CT. Deoarece baza de date utilizată în lucrarea de față conține exclusiv imagini grayscale.

Rezultatele raportate în [15] sunt consistente pe patru aplicații medicale diferite. La date complete, fine-tuning adecvat depășește sau egalează antrenarea de la zero. La date reduse, diferența devine semnificativă: la 25% din datele de antrenament pentru detecția polipilor, modelul antrenat de la zero suferă o degradare dramatică, în timp ce cel fine-tuned menține o performanță ridicată. La 1% din date pentru clasificarea frame-urilor de colonoscopie, modelul de la zero colapsează, iar cel fine-tuned rămâne utilizabil. Concluzia autorilor este directă: fine-tuning-ul trebuie să fie întotdeauna opțiunea preferată, indiferent de dimensiunea setului de antrenament disponibil. [15]



Figură 3.5 Analiza FROC pentru detecția polipilor, comparând șase variante de antrenare. [15]

Figura 3.5 [15] prezintă analiza FROC pentru detecția polipilor, comparând șase variante de antrenare. Curba neagră, corespunzătoare fine-tuning-ului complet, domină clar la rate mici de fals pozitiv, unde diferența față de celelalte variante este cea mai vizibilă. Modelul antrenat de la zero

(roz) are o comportare instabilă, cu o curbă care crește brusc abia la rate ridicate de fals pozitivi, confirmând că fără greutate pre-antrenate rețeaua necesită mult mai multe exemple false înainte de a atinge o sensibilitate comparabilă. Metoda clasică hand-crafted (galben) este net inferioară tuturor variantelor CNN. [15]

În contextul lucrării de față, encoderul ResNet50 este inițializat cu greutate pre-antrenate pe ImageNet. Imaginile de intrare sunt ecografii grayscale, adaptate pentru intrarea în trei canale prin replicarea canalului unic. Această strategie reduce semnificativ numărul de exemple necesare pentru convergență și oferă un punct de plecare mai bun față de inițializarea aleatoare, în condițiile în care setul de date disponibil este limitat - aprox. 1500 imagini.

3.5 Funcții de loss pentru segmentare (BCE, Dice, Tversky, Focal Tversky)

Funcția de pierdere folosită în antrenare influențează direct ce optimizează rețeaua. În segmentarea medicală, alegerea ei este critică din cauza dezechilibrului dintre structura de interes și fundalul imaginii.

Binary cross-entropy: BCE calculează pierderea independent pentru fiecare pixel, comparând probabilitatea prezisă cu eticheta binară. Problema apare când structura de interes ocupă o fracțiune mică din imagine: gradientii proveniți din pixelii de fundal domină complet optimizarea, iar rețeaua converge spre a prezice aproape tot fundal. Eroarea numerică este mică, dar structura clinică relevantă este ratată sau detectată parțial.

Dice loss: Milletari et al. [16] propun o funcție de pierdere bazată direct pe coeficientul Dice, definit ca:

$$D = \left(2 \sum_i p_i g_i \right) / \left(\sum_i p_i^2 + \sum_i g_i^2 \right)$$

Ecuția 3.1: Dice loss. [16]

unde p_i este predicția rețelei la pixelul i , iar g_i este eticheta ground truth. Numărătorul cuantifică intersecția dintre predicție și ground truth, iar numitorul aproximează suma cardinalităților celor două seturi. Spre deosebire de BCE, Dice loss calculează raportul de suprapunere la nivelul întregii imagini, ceea ce face ca fiecare pixel de foreground să aibă influență proporțională cu importanța lui în acoperirea structurii de interes. Consecința directă este că nu mai este necesară ponderarea manuală a claselor. [16]

Un avantaj suplimentar este coerența dintre antrenare și evaluare: Dice loss optimizează direct coeficientul Dice, care este și metrica standard de evaluare în segmentarea medicală. Diferența față de cross-entropy este demonstrată numeric în [16]: pe segmentarea prostatei în MRI, V-Net antrenat cu Dice loss obține un Dice mediu de 0.869, față de 0.739 al aceleiași arhitecturi antrenate cu logistic loss ponderat, o îmbunătățire de 17.6% relativă cu arhitectură identică.

Tversky loss. Dice loss penalizează egal false pozitivele și false negativele, deoarece este media armonică dintre precizie și recall. Salehi et al. [17] arată că în detecția leziunilor mici, numărul de pixeli non-leziune depășește de peste 500 de ori numărul de pixeli de leziune, iar antrenarea cu Dice simplu produce rețele cu precizie ridicată dar recall scăzut: leziunile sunt ratate sistematic.

Tversky loss generalizează Dice prin introducerea a doi parametri care controlează separat penalizarea celor două tipuri de eroare:

$$T(\alpha, \beta) = \left(\sum_i p_{0i} g_{0i} \right) / \left(\sum_i p_{0i} g_{0i} + \alpha \sum_i p_{0i} g_{1i} + \beta \sum_i p_{1i} g_{0i} \right)$$

Ecuația 3.2: Tversky loss. [17]

Termenul α penalizează false positives, iar β penalizează false negatives. Când $\alpha = \beta = 0.5$, Tversky loss se reduce la Dice loss. Pentru structuri mici, se folosește $\beta > \alpha$, astfel încât rețeaua să penalizeze mai mult leziunile ratate decât predicțiile eronate. [17]

Rezultatele raportate în [17] pe segmentarea leziunilor de scleroză multiplă confirmă această ipoteză. Configurația $\alpha=0.3$, $\beta=0.7$ obține un DSC de 56.42% și o sensibilitate de 56.85%, față de 53.42% DSC și 49.85% sensibilitate pentru Dice simplu ($\alpha=\beta=0.5$). Configurațiile cu β mai mare cresc monoton sensibilitatea, dar DSC are un maxim la $\beta=0.7$, după care scade din cauza acumulării de false positives.

Focal Tversky loss. Abraham și Khan (2019) [31] extind Tversky loss prin adăugarea unui parametru γ , analogă Focal Loss din detecția de obiecte:

$$FTL = (1 - T(\alpha, \beta))^{(1/\gamma)}$$

Ecuația 3.3: Focal Tversky loss. [31]

Parametrul γ reglează cât de mult contribuie exemplele ușor de clasificat față de cele greu de segmentat. Pentru structuri mici cu margini neclare, $\gamma > 1$ concentrează antrenarea pe cazurile dificile, reducând influența pixelilor de fundal care sunt clasificați corect cu ușurință. [31]

În lucrarea de față, funcția de pierdere combinată folosită este o sumă ponderată între BCE și Focal Tversky, cu ponderi diferite per structură, pentru a compensa dezechilibrele severe de scară dintre ficat, stomac, venă ombilicală și artera aortică abdominală.

3.6 Metrice de evaluare (Dice, IoU, sensibilitate, specificitate, precizie)

Evaluarea calității unei segmentări se face prin compararea măștii prezise de rețea cu masca ground truth. Taha și Hanbury [22] formalizează această comparație prin matricea de confuzie, care împarte pixelii imaginii în patru categorii: TP (prezis pozitiv, corect), FP (prezis pozitiv, incorect), FN (prezis negativ, incorect) și TN (prezis negativ, corect). [22]

Coeficientul Dice este metrica cea mai utilizată în validarea segmentărilor medicale. Măsoară suprapunerea dintre predicție și ground truth, normalizată față de suma cardinalităților celor două seturi:

$$DICE = \frac{(2TP)}{(2TP + FP + FN)}$$

Ecuatia 3.4 Formula Dice. [22]

IoU, numit și indicele Jaccard, măsoară intersecția împărțită la reuniunea celor două segmentări [22]:

$$IOU = \frac{(TP)}{(TP + FP + FN)}$$

Ecuatia 3.5 Formula IOU. [22]

Cele două metrici sunt legate matematic prin relația $IOU = DICE / (2 - DICE)$, ceea ce înseamnă că produc același ranking al modelelor și nu aduc informație suplimentară dacă sunt raportate împreună. IoU este întotdeauna mai mic decât Dice, cu excepția valorilor extreme 0 și 1. [22]

Sensitivitatea măsoară proporția pixelilor pozitivi din ground-truth identificați corect de model [22]:

$$SENS = \frac{(TP)}{(TP + FN)}$$

Ecuatia 3.6 Formula SENS. [22]

Specificitatea măsoară proporția pixelilor de fundal clasificați corect [22]:

$$SPEC = \frac{(TN)}{(TN + FP)}$$

Ecuatia 3.7 Formula SPEC. [22]

Precizia măsoară câți dintre pixelii prezisi pozitiv sunt cu adevărat pozitivi în ground truth [22]:

$$PREC = \frac{(TP)}{(TP + FP)}$$

Ecuatia 3.8 Formula PREC. [22]

În segmentarea multi-organ, metricile se raportează separat per structură. Un scor agregat ar reflecta în principal performanța pe structura dominantă volumetric și ar masca eșecuri complete pe structuri mai mici. [22] În cazul lucrării de față, ficatul și stomacul ocupă suprafețe semnificativ mai

mari decât vena ombilicală sau artera aortică abdominală; un scor mediu ar ascunde comportamentul rețelei pe structurile mici, care sunt și cele mai dificil de segmentat.

4 Setul de date și preprocesarea

4.1 Descrierea setului de date

Setul de date folosit în această lucrare este "Fetal Abdominal Structures Segmentation Dataset Using Ultrasonic Images", publicat pe platforma Mendeley Data de Da Correggio et al. în octombrie 2023, cu DOI 10.17632/4gcpm9dsc3.1. [23]

Datele au fost colectate între septembrie 2021 și septembrie 2023 la Maternitatea Spitalului Universitar Polydoro Ernani de São Thiago din Florianópolis, Santa Catarina, Brazilia. Studiul a inclus 169 de subiecți, de la care au fost obținute în total 1588 de imagini ecografice ale circumferinței abdominale fetale. [23]

Criteriile de includere au vizat femei gravide la termen, cu vârsta de cel puțin 18 ani, aflate în travaliu sau programate pentru naștere. Au fost incluse și paciente cu complicații precum diabet gestațional, preeclampsie, ruptura membranelor sau restricție de creștere intrauterină. Au fost excluși fătii prematuri, sarcinile multiple și cazurile cu anomalii structurale sau cromozomiale fetale. [23]

Studiul a primit aprobarea Comitetului de Etică în Cercetare al Universității Federale Santa Catarina, cu numărul de aprobare 4.971.754, iar toți participanții au semnat consimțământul informat. [23]

Imaginile au fost achiziționate cu echipamente Siemens Acuson, Voluson 730 (GE Healthcare) sau Philips-EPIQ Elite (Philips Healthcare), cu transductori curbiliniari de 2-9 MHz. Protocolul de achiziție a fost standardizat: s-a capturat o secțiune axială la nivelul celei mai largi porțiuni a abdomenului fetal, care cuprinde ficatul, stomacul, coloana vertebrală, artera aortică abdominală și porțiunea intrahepatică a venei ombilicale. Imaginile au fost stocate în format DICOM. [23]

Un proces de control al calității a exclus imaginile cu calipers vizibili sau cu umbre acustice evidente din structurile osoase. Adnotarea structurilor a fost realizată de clinicieni experți folosind software-ul 3D Slicer, versiunea 5.2.2. Cele patru structuri adnotate sunt: artera aortică abdominală, vena ombilicală intrahepatică, stomacul fetal și ficatul fetal. [23]



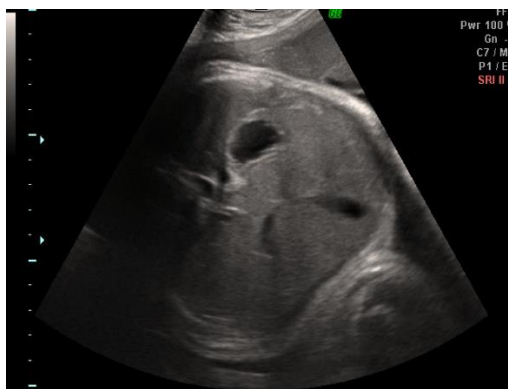
Figură 4.1 Exemplu de imagine din setul de date [23]

4.2 Formatul datelor și organizarea fișierelor

Dataset-ul este organizat în două foldere paralele: IMAGES și ARRAY_FORMAT. Ambele conțin câte 1.588 de fișiere, câte unul per imagine, cu denumiri identice și extensii diferite. [23]

Convenția de denumire este uniformă: fiecare fișier este identificat prin prefixul pacientului și numărul imaginii, de forma P{XXX}_IMG{Y}. De exemplu, P01_IMG1 corespunde primei imagini a primului pacient, iar P02_IMG3 celei de-a treia imagini a celui de-al doilea pacient. Numărul de imagini per pacient variază de la un subiect la altul.

Folderul IMAGES conține imaginile ecografice brute în format PNG, grayscale, cu dimensiunea de 768x1024 pixeli. Imaginile includ textul suprapus de echipamentul de achiziție în marginile stângi și drepte, precum și o bordură neagră în jurul zonei utile.



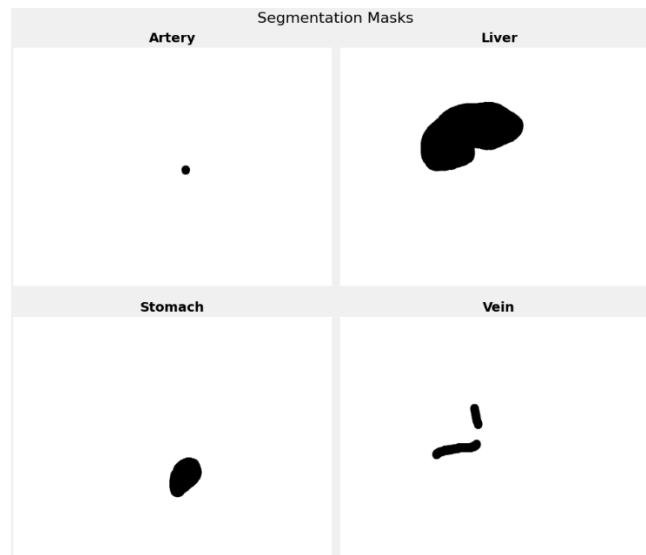
Figură 4.2 P016_IMG7 [23]

Folderul ARRAY_FORMAT conține fișiere NPY, formatul binar al bibliotecii NumPy. Fiecare fișier stochează un dicționar Python cu două chei. Prima cheie, "image", conține imaginea ecografică sub formă de array numpy cu shape (768, 1024) și valori uint8 în intervalul 0-255. A doua cheie, "structures", conține la rândul ei un dicționar cu patru chei: "artery", "liver", "stomach" și "vein", fiecare reprezentând masca binară de segmentare a structurii respective, cu același shape (768, 1024) și valori 0 sau 1.

```
Structura detaliată a dicționarului:
Dict cu 2 chei:
  'image': numpy array, shape=(768, 1024), dtype=uint8, min=0.0000, max=255.0000
  'structures':
    Dict cu 4 chei:
      'artery': numpy array, shape=(768, 1024), dtype=uint8, min=0.0000, max=1.0000
      'liver': numpy array, shape=(768, 1024), dtype=uint8, min=0.0000, max=1.0000
      'stomach': numpy array, shape=(768, 1024), dtype=uint8, min=0.0000, max=1.0000
      'vein': numpy array, shape=(768, 1024), dtype=uint8, min=0.0000, max=1.0000
```

Figură 4.3 Structura fiecărui element din setul de date

Măștile sunt binare: valoarea 1 indică pixelii care aparțin structurii, iar valoarea 0 indică fundalul. Diferența de scară dintre structuri este vizibilă direct în măști: ficatul ocupă o suprafață semnificativă din imagine, în timp ce masca arterei aortice se reduce uneori la câțiva zeci de pixeli.



Figură 4.4 Cele 4 măști din P01_IMG01 [23]

4.3 Împărțirea train/val/test la nivel de pacient

Împărțirea setului de date s-a făcut la nivel de pacient, nu la nivel de imagine. Această decizie evită scurgerea de informație între subseturi: dacă imaginile aceluiași pacient ar fi distribuite atât în train cât și în test, rețeaua ar fi evaluată parțial pe date pe care le-a văzut în antrenare, sub o altă formă. Prin împărțirea la nivel de pacient, subsetul de test conține exclusiv pacienți pe care modelul nu i-a văzut în nicio etapă a antrenării.

Concret, din cei 169 de pacienți disponibili, s-au extras identificatorii unici din numele fișierelor, după convenția $P\{XX\}_{IMG}\{YY\}$. Lista de pacienți a fost împărțită aleator în proporțiile 70/15/15, folosind funcția `train_test_split` din biblioteca `scikit-learn` cu `seed` fix 42 pentru reproductibilitate. Toate imaginile unui pacient au fost alocate integral subsetului corespunzător. [24]

Proporția 70/15/15 reprezintă o împărțire standard în literatura de specialitate pentru seturi de date de dimensiune medie. Subsetul de validare a fost folosit exclusiv pentru monitorizarea performanței în timpul antrenării, fără a influența actualizarea greutăților. Subsetul de test a fost folosit o singură dată, la finalul antrenării, pentru evaluarea finală a modelului.

Deoarece numărul de imagini per pacient variază, împărțirea la nivel de pacient nu produce proporții exacte la nivel de imagini. Distribuția reală a ieșit 71.2/13.9/14.9, ceea ce corespunde unui total de 1.130 imagini de antrenare, 221 de validare și 236 de test, din totalul de 1.588 de imagini ale dataset-ului. [23]

4.4 Augmentarea datelor (Albumentations)

Augmentarea datelor este o tehnică prin care setul de antrenare este extins artificial prin aplicarea unor transformări aleatorii asupra imaginilor existente. Scopul este de a expune rețeaua la variații pe care le-ar putea întâlni în date reale, fără a necesita imagini adnotate suplimentare. În această lucrare, augmentarea a fost realizată folosind biblioteca `Albumentations` [25], aplicată

exclusiv pe subsetul de antrenare. Subseturile de validare și test primesc doar redimensionare și normalizare, pentru a asigura o evaluare consistentă și neinfluențată de transformări aleatorii.

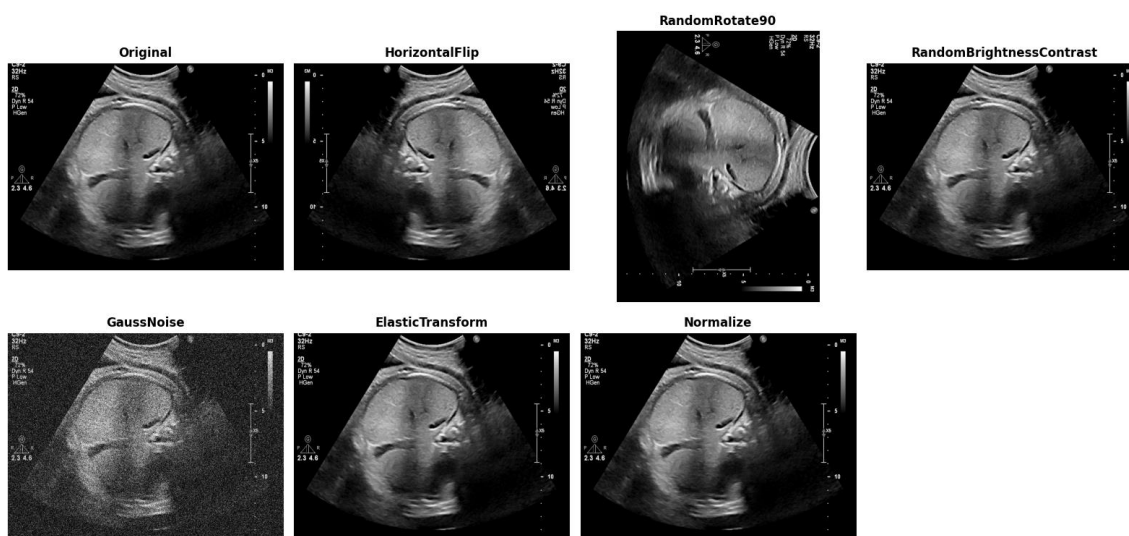
Prima operație aplicată tuturor subseturilor este redimensionarea imaginilor de la rezoluția originală de 768x1024 pixeli la 256x256 pixeli. Această decizie reduce costul computațional al antrenării și uniformizează dimensiunea intrării în rețea, cerință impusă de arhitectura fixă a encoderului ResNet50.

HorizontalFlip [25] oglindește imaginea față de axa verticală, cu probabilitatea 0.5. În imagistica ecografică abdominală, structurile fetale nu au o orientare fixă față de transductor: poziția fătului variază de la o examinare la alta. Această transformare simulează tocmai această variabilitate, fără a introduce distorsiuni anatomice.

RandomRotate90 [25] aplică o rotație aleatoare de 90 de grade, cu probabilitatea 0.5. Similar cu flip-ul orizontal, rotațiile sunt justificate de faptul că secțiunea abdominală fetală poate fi capturată din unghiuri diferite în funcție de poziționarea transductorului și a pacientei.

RandomBrightnessContrast [25] modifică aleator luminozitatea și contrastul imaginii în intervalul ± 0.2 , cu probabilitatea 0.5. Imaginile ecografice sunt puternic influențate de setările echipamentului, de adâncimea de penetrare și de caracteristicile țesuturilor fiecărui pacient. Variațiile de luminozitate și contrast simulate de această transformare reproduc o parte din această variabilitate inter-pacient și inter-echipament.

GaussNoise [25] adaugă zgomot gaussian aleator asupra imaginii, cu probabilitatea 0.3. Zgomotul este o caracteristică intrinsecă a imaginilor ecografice, cauzat de fenomenul de speckle, specific undelor ultrasonice. Expunerea rețelei la imagini cu zgomot adăugat artificial îi crește robustețea față de variațiile de calitate a imaginii întâlnite în practică.



Figură 4.5 Toate augmentările, aplicate unei imagini [23]

ElasticTransform [25] aplică deformări elastice locale asupra imaginii, cu parametrii $\alpha=1.0$ și $\sigma=50.0$ și probabilitatea 0.2. Deformările elastice modifică local geometria imaginii, simulând variațiile de formă ale țesuturilor moi. Ronneberger et al. [9] folosesc același tip de augmentare în lucrarea originală U-Net, argumentând că deformările elastice sunt cea mai frecventă și naturală variație întâlnită în imagistica biologică.

Toate transformările geometrice, HorizontalFlip, RandomRotate90 și ElasticTransform, se aplică simultan și consistent atât imaginii, cât și celor patru măști de segmentare, pentru a păstra

corespondența spațială dintre ele. O transformare aplicată doar imaginii, fără a fi reflectată în măști, ar introduce erori de supervizare în antrenare.

La final, imaginile sunt normalizate cu media 0.5 și deviația standard 0.5 și convertite în tensori PyTorch prin operatorul ToTensorV2. [25]

4.5 Adaptarea single-channel pentru intrarea ResNet50

Encoderul ResNet50 a fost conceput pentru imagini color RGB, adică pentru intrări cu trei canale. Primul strat convoluțional, conv1, are prin urmare greutatea de dimensiune 64x3x7x7: 64 de filtre, fiecare cu 3 planuri de adâncime corespunzătoare canalelor roșu, verde și albastru. Imaginile ecografice sunt grayscale, cu un singur canal, ceea ce face incompatibilă intrarea implicită a rețelei cu datele din această lucrare.

Tajbakhsh et al. [15] rezolvă această problemă prin repetarea canalului gri de trei ori, producând un tensor de trei canale identice care mimează structura unui input RGB. Avantajul este că primul strat convoluțional rămâne nemodificat structural, iar greutatea pre-antrenată este folosită integral fără nicio ajustare. Dezavantajul este că cele trei canale sunt identice, deci fiecare dintre cele 64 de filtre aplică aceeași operație pe același semnal de trei ori, ceea ce introduce redundanță în primul strat. [15]

În această lucrare s-a ales o abordare diferită, documentată de Hughes [26]. Primul strat convoluțional este înlocuit cu unul nou, cu un singur canal de intrare, iar greutatea sa este inițializată prin suma canalelor RGB ale greutăților originale pre-antrenate:

```
weights = resnet.conv1.weight.clone()
resnet.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False)
resnet.conv1.weight.data = torch.sum(weights, dim=1, keepdim=True)
```

Prin această operație, fiecare dintre cele 64 de filtre primește o singură greutate per poziție spațială, obținută prin sumarea contribuțiilor celor trei canale originale. Rezultatul este că informația din greutatea pre-antrenată pe ImageNet este păstrată și comprimată într-un singur canal [26], în loc să fie ignorată sau replicată. Rețeaua pornește astfel dintr-un punct de inițializare informat, nu dintr-unul aleator, chiar dacă modalitatea de intrare s-a schimbat. [26]

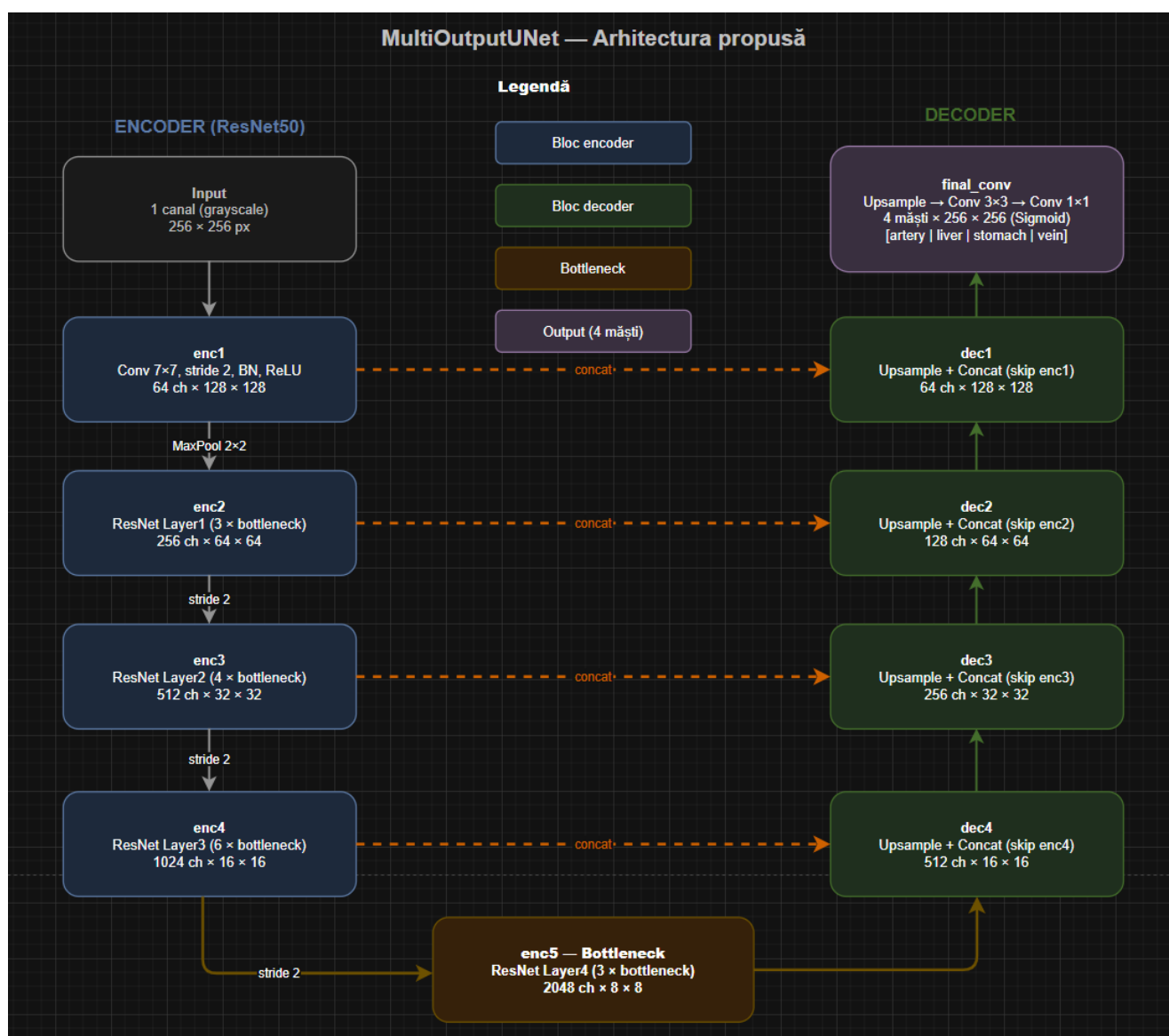
Această alegere are un avantaj practic direct: primul strat operează pe o singură copie a semnalului, fără redundanță, iar greutatea inițială reflectă o combinație liniară a reprezentărilor RGB originale, care captează aproximativ aceeași informație de luminanță pe care o conține și imaginea grayscale.

5 Arhitectura propusă (MultiOutputUNet)

5.1 Schema generală a arhitecturii

Arhitectura propusă se numește MultiOutputUNet și este construită pe structura U-Net [9], cu encoderul înlocuit de ResNet50 pre-antrenat [10]. Intrarea este o imagine ecografică grayscale de 256×256 pixeli. Ieșirea constă în patru măști binare de segmentare, câte una pentru fiecare structură: artera aortică abdominală, vena ombilicală intrahepatică, ficatul fetal și stomacul fetal. Schema arhitecturii este prezentată în Figura 5.1.

Rețeaua are trei componente principale: encoderul, bottleneck-ul și decoderul.



Figură 5.1 Arhitectura modelului propus în lucrare

Encoderul comprimă imaginea de intrare în reprezentări din ce în ce mai abstracte, reducând rezoluția spațială și crescând numărul de canale. Este format din cinci stagii. enc1 aplică o convoluție 7×7 cu stride 2, Batch Normalization și ReLU, producând feature maps de 64 de canale la rezoluția 128×128 . Între enc1 și enc2 se aplică MaxPool 2×2 . enc2, enc3 și enc4 corespund primelor trei grupuri de blocuri reziduale din ResNet50 și produc 256, 512 și respectiv 1024 de canale, la rezoluțiile

64×64 , 32×32 și 16×16 . Downsampling-ul intern se face prin stride 2 în primul bloc al fiecărui stagi. $enc5$ este bottleneck-ul rețelei: produce 2048 de canale la rezoluția minimă de 8×8 pixeli. La acest nivel rețeaua are o vedere globală asupra imaginii, dar nu mai știe unde anume se află structurile în spațiu.

Decoderul reconstruiește rezoluția prin patru blocuri: $dec4$, $dec3$, $dec2$ și $dec1$. La fiecare nivel, feature maps din decoder sunt făcute upsamples bilinear cu factorul 2, apoi concatenate cu feature maps corespunzătoare din encoder prin skip connections. Acest mecanism este important în cazul structurilor mici, cum este artera aortică: informația despre poziția și conturul ei se pierde rapid în encoder prin compresie, iar skip connections o recuperează direct din stagiile timpurii ale encoderului, unde rezoluția era încă ridicată. Două convoluții 3×3 cu Batch Normalization și ReLU prelucrează reprezentarea concatenată la fiecare nivel. Rezoluția crește de la 8×8 la 128×128 , iar numărul de canale scade de la 512 la 64.

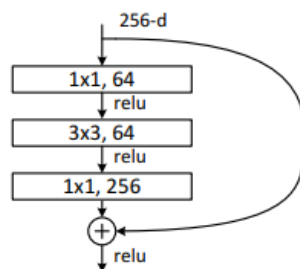
Stratul final face upsamples la 256×256 , aplică o convoluție 3×3 urmată de ReLU și o convoluție 1×1 care produce patru canale de ieșire. Sigmoid aplicat per canal transformă logits-urile în probabilități per pixel. Pragul de decizie la inferență este 0.5. Cele patru canale sunt procesate independent, fiecare reprezentând probabilitatea ca un pixel să aparțină structurii corespunzătoare. Problema este tratată ca segmentare multi-label: un pixel poate fi prezis pozitiv pentru mai multe structuri simultan, ceea ce în practică apare rar, dar arhitectura nu impune nicio constrângere de excludere mutuală între clase.

5.2 Encoderul bazat pe ResNet50 modificat

Encoderul arhitecturii este ResNet50 pre-antrenat pe ImageNet [10], modificat pentru a accepta imagini cu un singur canal, conform descrierii din subcapitolul 4.5. Alegerea ResNet50 ca encoder este motivată de două avantaje principale: arhitectura sa reziduală rezolvă problema degradării în rețele adânci, iar greutatea pre-antrenate pe ImageNet oferă un punct de plecare mai bun decât inițializarea aleatoare, chiar și pentru imagini ecografice.

Primul stagi, $enc1$, conține convoluția inițială de 7×7 cu stride 2, Batch Normalization și ReLU. Rezoluția imaginii scade de la 256×256 la 128×128 , cu 64 de canale la ieșire. Câmpul receptiv mare al convoluției 7×7 permite capturarea de structuri la scară mai mare încă din primul stagi, util în contextul imaginilor ecografice unde ficatul ocupă o suprafață semnificativă din imagine. Imediat după $enc1$ se aplică MaxPool 2×2 , care reduce rezoluția la 64×64 înainte de a intra în $enc2$.

$enc2$, $enc3$, $enc4$ și $enc5$ corespund celor patru grupuri de blocuri reziduale din ResNet50: $layer1$, $layer2$, $layer3$ și respectiv $layer4$, cu 3, 4, 6 și 3 blocuri bottleneck. Fiecare bloc bottleneck este format din trei convoluții consecutive: o convoluție 1×1 care reduce numărul de canale, o convoluție 3×3 care operează la dimensionalitate redusă și o convoluție 1×1 care restaurează numărul de canale inițial. Batch Normalization este aplicat intern după fiecare convoluție din bloc. Conexiunile reziduale din fiecare bloc adună ieșirea celor trei convoluții cu intrarea originală a blocului, permițând gradientilor să se propage eficient înapoi prin rețea în timpul antrenării. Downsampling-ul între stagii se face prin stride 2 în primul bloc al lui $layer2$, $layer3$ și $layer4$, fără pooling explicit.



Figură 5.2 Structura unui bloc bottleneck din ResNet50. Adaptat după He et al. [10].

La ieșirea fiecărui stadiu, dimensiunile feature maps sunt: enc2 — 256 canale, 64×64 ; enc3 — 512 canale, 32×32 ; enc4 — 1024 canale, 16×16 ; enc5 — 2048 canale, 8×8 . Numărul de canale se dublează la fiecare stadiu pe măsură ce rezoluția se înjumătățește, un compromis de design standard în arhitecturile convoluționale adânci [10].

Toți parametrii encoderului s-au actualizat liber în antrenare, fără nicio înghețare de straturi. Tajbakhsh et al. [15] arată că fine-tuning-ul complet produce rezultate mai bune decât înghețarea straturilor timpurii atunci când datele țintă diferă semnificativ de ImageNet. Imaginile ecografice sunt suficient de diferite față de imaginile naturale din ImageNet pentru ca această regulă să se aplice: texturile, contrastele și structurile vizibile într-o ecografie nu au corespondent direct în fotografiile naturale.

Feature maps produse de enc1, enc2, enc3 și enc4 sunt reținute și transmise decoderului prin skip connections, la nivelul de rezoluție corespunzător. enc5 nu produce skip connection, ieșirea lui fiind transmisă direct primului bloc al decoderului. Aceasta este singura diferență față de U-Net original [9], unde toate nivelurile encoderului produc skip connections, inclusiv cel mai adânc.

5.3 Blocurile decoder cu skip connections

Decoderul este format din patru blocuri identice ca structură: dec4, dec3, dec2 și dec1. Fiecare bloc primește două intrări: ieșirea blocului anterior din decoder și feature maps corespunzătoare din encoder prin skip connection. Clasa care implementează un bloc decoder se numește DecoderBlock și este definită separat, fiind instanțiată de patru ori cu parametri diferiți de canale.

Primul pas din fiecare bloc este upsamplul bilinear cu factorul 2, care dublează rezoluția spațială a feature maps primite de la blocul anterior. Upsamplul bilinear interpolează valorile pixelilor lipsă pe baza vecinilor, producând o tranziție mai lină decât transposed convolution. Dacă după upsampl dimensiunile spațiale nu se potrivesc exact cu cele ale skip connection-ului din encoder, se aplică o interpolare suplimentară pentru aliniere. Această situație poate apărea din cauza convoluțiilor fără padding din encoder, care reduc ușor dimensiunile spațiale.

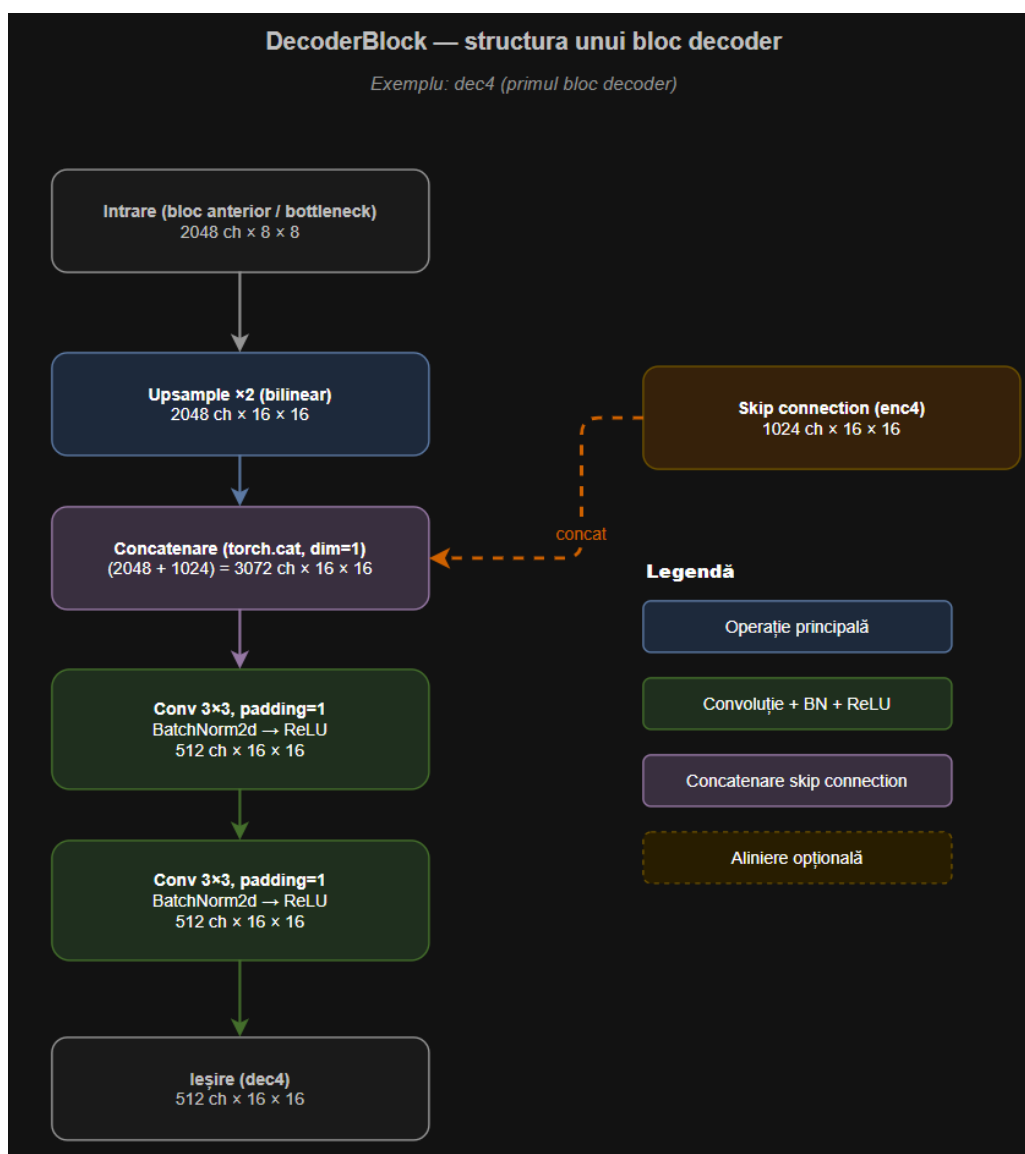
După upsampl, feature maps sunt concatenate cu skip connection-ul corespunzător din encoder de-a lungul axei canalelor. Concatenarea combină informația semantică bogată din decoder cu detaliile spațiale fine din encoder, la rezoluția respectivă. Numărul de canale după concatenare este suma canalelor celor două intrări: de exemplu, la dec4, $2048 \times 8 \times 8$ de canale din enc5 devin $2048 \times 16 \times 16$ după upsampl, la care se adaugă 1024 de canale din enc4, rezultând 3072 de canale la intrarea în convoluții.

Urmează două convoluții 3×3 consecutive, fiecare urmată de Batch Normalization și ReLU. Aceste convoluții prelucrează reprezentarea concatenată și produc feature maps cu numărul de canale specificat pentru blocul respectiv. Parametrii celor patru blocuri decoder sunt: dec4 primește 2048

canale din bottleneck și 1024 din enc4, producând 512 canale la 16×16 ; dec3 primește 512 din dec4 și 512 din enc3, producând 256 canale la 32×32 ; dec2 primește 256 din dec3 și 256 din enc2, producând 128 canale la 64×64 ; dec1 primește 128 din dec2 și 64 din enc1, producând 64 canale la 128×128 .

Skip connections din această arhitectură funcționează diferit față de cele din ResNet [10]. În ResNet, conexiunile reziduale adună element-wise ieșirea unui bloc cu intrarea sa, în cadrul aceluiași stadiu. În MultiOutputUNet, skip connections leagă niveluri diferite de rezoluție între encoder și decoder și combină prin concatenare, nu prin adunare. Diferența este importantă: concatenarea păstrează ambele reprezentări intacte și lasă convoluțiile următoare să decidă cum să le combine, în timp ce adunarea le fuzionează direct.

Figura 5.3 prezintă structura unui bloc DecoderBlock, ilustrată pentru dec4. Intrarea din blocul anterior, cu 2048 de canale la rezoluția 8×8 , se face mai întâi upsamplare bilinear cu factorul 2, ajungând la rezoluția 16×16 . Skip connection-ul din enc4, cu 1024 de canale la aceeași rezoluție 16×16 , este concatenat cu ieșirea de la upsamplare de-a lungul axei canalelor, producând un tensor de 3072 de canale. Cele două convoluții 3×3 consecutive, fiecare urmată de Batch Normalization și ReLU, reduc numărul de canale la 512 și rafinează reprezentarea combinată. Ieșirea blocului dec4 este un tensor de 512 canale la rezoluția 16×16 , care devine intrare pentru dec3.



Figură 5.3 Structura unui bloc Decoder

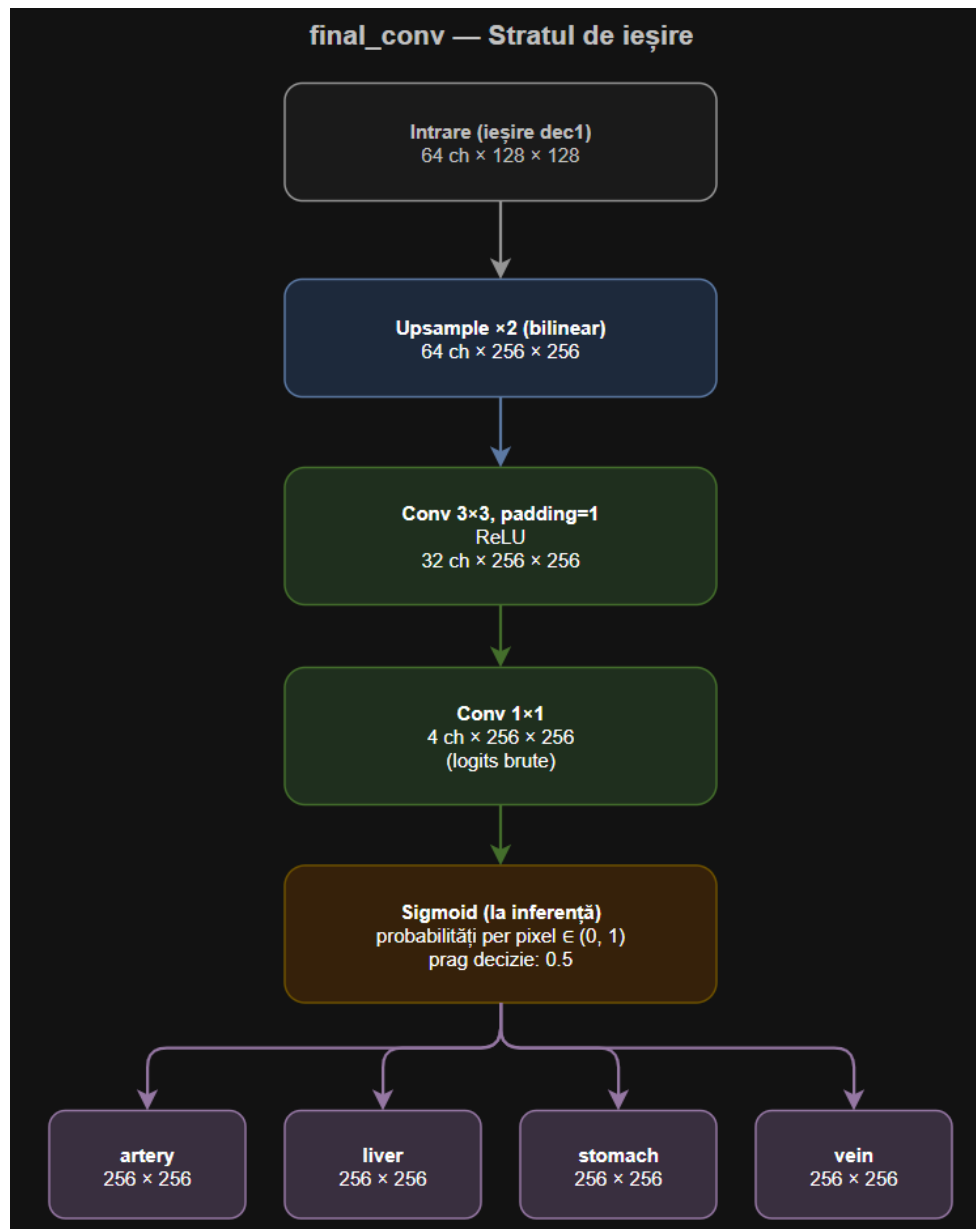
5.4 Stratul final și ieșirile multi-organ

Stratul final, denumit `final_conv`, preia ieșirea lui `dec1`, care are 64 de canale la rezoluția 128×128 , și produce patru hărți de activare la rezoluția originală a imaginii de intrare, 256×256 pixeli.

Prima operație este un `upsample bilinear` cu factorul 2, care restaurează rezoluția de la 128×128 la 256×256 . Urmează o convoluție 3×3 cu padding 1, care reduce numărul de canale de la 64 la 32, urmată de `ReLU`. Această convoluție rafinează reprezentarea înainte de predicția finală, fără a modifica rezoluția spațială. Ultima operație este o convoluție 1×1 care mapează cei 32 de canale la exact patru canale de ieșire, câte unul pentru fiecare structură anatomică: artera aortică abdominală, vena ombilicală intrahepatică, ficatul fetal și stomacul fetal. Ordinea canalelor este alfabetică, impusă de modul în care biblioteca NumPy sortează cheile dicționarului din fișierele NPY: `artery`, `liver`, `stomach`, `vein`.

Rețeaua nu aplică sigmoid în interiorul arhitecturii. La antrenare, logits-urile brute sunt transmise direct funcției de pierdere, care internalizează sigmoid prin `BCEWithLogitsLoss` pentru stabilitate numerică. La inferență, sigmoid este aplicat explicit pe ieșirile rețelei, transformând logits-urile în probabilități per pixel în intervalul (0, 1). Pragul de decizie pentru binarizarea măștilor este 0.5: pixelii cu probabilitate peste acest prag sunt clasificați ca aparținând structurii respective, ceilalți ca fundal.

Cele patru canale sunt procesate complet independent: nu există nicio constrângere care să impună excludere mutuală între clase. Un pixel poate fi prezis pozitiv pentru mai multe structuri simultan, ceea ce arhitectura permite prin natura formulării multi-label. În practică, suprapunerile anatomice reale între structurile segmentate sunt rare, însă arhitectura nu le exclude din principiu.



Figură 5.4 Stratul final din rețea

5.5 Funcția de loss combinată

Funcția de pierdere folosită în antrenare combină două componente: Binary Cross-Entropy ponderat per organ și Focal Tversky loss. Combinarea lor este motivată de natura dezechilibrată a problemei: structurile segmentate diferă dramatic ca suprafață, iar o singură funcție de pierdere nu gestionează bine simultan structuri mari și structuri mici.

Prima componentă este BCEWithLogitsLoss, implementată cu ponderi pozitive diferite per organ. Ponderile sunt: 150.0 pentru arteră, 15.0 pentru ficat, 20.0 pentru stomac și 30.0 pentru venă. Aceste valori reflectă dezechilibrul dintre suprafața ocupată de fiecare structură și fundalul imaginii: artera aortică este cea mai mică structură din dataset și primește ponderea cea mai mare, pentru a forța rețeaua să nu o ignore în favoarea structurilor dominante. BCEWithLogitsLoss combină intern sigmoid cu cross-entropy, evitând instabilitatea numerică apărută la calculul separat al celor două operații. Milletari et al. [16] arată că funcțiile de pierdere bazate pe suprapunere sunt superioare cross-

entropy simplu tocmai în scenarii cu dezechilibru sever între foreground și background, motiv pentru care BCE singur nu este suficient. [16]

A doua componentă este Focal Tversky loss [17]. Parametrii folosiți sunt $\alpha=0.4$, $\beta=0.6$ și $\gamma=0.75$. Valoarea $\beta > \alpha$ penalizează mai mult false negatives decât false positives, orientând antrenarea spre un recall mai ridicat. Salehi et al. [17] demonstrează că pentru structuri mici, configurații cu $\beta > \alpha$ produc rezultate superioare față de Dice simplu ($\alpha=\beta=0.5$), prin reducerea sistematică a false negatives. Parametrul $\gamma=0.75$, subunitar, reduce influența exemplelor deja bine clasificate și concentrează optimizarea pe pixelii dificili, în special pe marginile structurilor și pe structurile mici. Focal Tversky este calculat per imagine și per organ, iar valoarea finală este media pe toate organele și toate imaginile din batch. [17]

Funcția de pierdere totală este:

$$L = \frac{(2)}{(5)}L_B + \frac{(3)}{(5)}L_F$$

Ecuția 5.1 Formula Funcției de loss

Ponderea mai mare acordată componentei Focal Tversky față de BCE reflectă prioritatea dată recall-ului față de precizie în contextul segmentării structurilor mici. O leziune sau o structură ratată complet este mai costisitoare decât un fals pozitiv, care poate fi corectat prin post-procesare. [16] [17]

6 Procesul de antrenare

6.1 Configurația experimentală (hiperparametri, optimizator, scheduler)

Antrenarea a fost realizată pentru 150 de epoci, cu un batch size de 8. Optimizatorul folosit este Adam, cu rata de învățare inițială de 3×10^{-4} și weight decay de 10^{-5} . Adam a fost ales față de SGD pentru că converge mai rapid pe seturi de date de dimensiune medie și necesită mai puțină ajustare manuală a ratei de învățare.

Scheduler-ul folosit este CosineAnnealingLR, cu T_max egal cu numărul total de epoci. Acesta reduce rata de învățare urmând o curbă cosinus de la valoarea inițială până aproape de zero pe parcursul antrenării, fără pași bruschi. Efectul practic este că în primele epoci rețeaua explorează mai agresiv spațiul parametrilor, iar spre final face ajustări fine în jurul unui minim.

Pragul de decizie pentru binarizarea măștilor la evaluare este 0.5, aplicat după sigmoid pe logits-urile rețelei. Numărul de workers pentru încărcarea datelor este 2, cu pin_memory activat pentru transfer mai rapid între RAM și memoria GPU. Seed-ul fix 42 a fost setat pentru NumPy, PyTorch și modulul random, asigurând reproductibilitatea împărțirii datelor și a augmentărilor.

Tabelul de mai jos rezumă hiperparametrii principali ai configurației experimentale:

Hiperparametru	Valoare
Număr epoci	150
Batch size	8
Optimizator	Adam
Rată de învățare	3×10^{-4}
Weight decay	10^{-5}
Scheduler	CosineAnnealingLR
T_max	150
Prag decizie	0.5
Seed	42

Tabel 6.1 Hiperparametrii folosiți pentru antrenare

6.2 Precizie mixtă și stabilizarea antrenării

Antrenarea cu precizie mixtă (Mixed Precision Training) este o tehnică prin care operațiile din rețea sunt executate alternativ în precizie redusă (float16) și precizie completă (float32) [27], în funcție de sensibilitatea lor numerică. Operațiile costisitoare computațional, cum sunt convoluțiile și înmulțirile de matrici, sunt executate în float16, reducând consumul de memorie și accelerând calculul pe GPU-urile moderne care au unități hardware dedicate pentru această precizie. Operațiile sensibile numeric, cum este calculul funcției de pierdere, rămân în float32 pentru a evita erorile de rotunjire. [27]

În implementare, precizia mixtă este gestionată prin `autocast` și `GradScaler` din PyTorch. `autocast` este aplicat în jurul propagării înainte și al calculului pierderii, permițând PyTorch să aleagă automat precizia optimă pentru fiecare operație. `GradScaler` scalează valoarea pierderii înainte de backpropagation pentru a preveni underflow-ul gradientilor în float16: gradientii foarte mici devin zero în float16 dacă nu sunt scalați în prealabil. După backpropagation, `GradScaler` descalează gradientii înainte de actualizarea parametrilor și verifică dacă au apărut valori infinite sau NaN; dacă da, pasul de optimizare este sărit pentru acel batch. [27]

Gradient clipping limitează norma gradientilor la o valoare maximă de 1.0 [28] înainte de fiecare pas de actualizare a parametrilor. Fără această limitare, gradientii pot crește exploziv în anumite batch-uri, în special în primele epoci de antrenare, producând actualizări masive ale parametrilor care destabilizează convergența. Clipping-ul nu elimină gradientii mari, ci le scalează proporțional astfel încât norma lor totală să nu depășească pragul setat. În cod, operația este aplicată după descalarea `GradScaler` și înainte de `optimizer.step()`. [28]

Combinarea celor două tehnici este standard în antrenarea rețelelor adânci pe GPU-uri cu memorie limitată. Precizia mixtă reduce la jumătate memoria ocupată de activări în timpul propagării înainte, permițând un batch size mai mare sau imagini de rezoluție mai mare la același consum de memorie. Gradient clipping asigură stabilitatea antrenării indiferent de variațiile din date. [27][28]

6.3 Strategia de salvare a checkpoint-urilor

Pe parcursul antrenării sunt salvate două tipuri de checkpoint-uri. Primul este un checkpoint de continuare, salvat la sfârșitul fiecărei epoci sub numele `last_checkpoint.pth`. Acesta conține starea completă a antrenării: greutatea modelului, starea optimizatorului, starea scheduler-ului, epoca curentă, cel mai bun Dice score de validare obținut până la acel moment și contorul de patience pentru early stopping. Scopul acestui checkpoint este de a permite reluarea antrenării de la ultimul punct în cazul unei întreruperi, fără a pierde progresul acumulat.

Al doilea tip este un checkpoint periodic, salvat la epocile 30, 60, 90, 120 și 150, sub numele `checkpoint_epoch_{N}.pth`. Aceste checkpoint-uri periodice permit compararea modelului din diferite momente ale antrenării și recuperarea unei versiuni anterioare dacă modelul final prezintă supraantrenare față de un punct intermediar.

La reluarea antrenării, flag-ul `RESUME_TRAINING` din configurație controlează dacă se încarcă `last_checkpoint.pth` sau se pornește de la zero. Dacă fișierul de checkpoint nu poate fi încărcat din cauza unui format incompatibil, se încearcă automat o a doua variantă de încărcare cu safe globals pentru scalari NumPy, problemă care apare uneori între versiuni diferite de PyTorch și NumPy.

Evaluarea finală a modelului se face pe checkpointul de la epoca 150, nu pe cel cu cel mai bun Dice de validare. Această decizie reflectă faptul că antrenarea a fost configurată fără early stopping activ, patience-ul fiind setat la 150, egal cu numărul total de epoci.

6.4 Mediul de antrenare

Antrenarea a fost realizată integral pe Google Colab, folosind un GPU NVIDIA A100. Nu s-a folosit nicio resursă de calcul locală. Setul de date a fost stocat pe Google Drive și copiat la începutul fiecărei sesiuni de antrenare în memoria RAM a instanței Colab prin dezarchivare locală, pentru a elimina latența accesului repetat la Drive în timpul încărcării batch-urilor.

Configurația software folosită este PyTorch 2.11.0 cu suport CUDA 12.8. Augmentarea datelor a fost realizată cu biblioteca Albumentations, iar încărcarea și preprocesarea datelor cu NumPy.

Antrenarea cu precizie mixtă a folosit modulul `torch.cuda.amp` din PyTorch, cu autocast și GradScaler.

O epocă de antrenare durează între 2 și 3 minute pe A100, în funcție de încărcarea serverului Colab. O rulare completă de 150 de epoci durează aproximativ 5 până la 7 ore. Testarea inițială a codului a fost făcută pe un GPU T4, disponibil în planul gratuit Colab, înainte de migrarea antrenării pe A100 pentru rulările complete.

Componentă	Detalii
Platformă	Google Colab
GPU antrenare	NVIDIA A100
GPU testare	NVIDIA T4
PyTorch	2.11.0+cu128
CUDA	12.8
Durata per epocă	2-3 minute
Durata totală (150 epoci)	~5-7 ore

Tabel 6.2 Detalii tehnice pentru antrenare

7 Experimente și rezultate

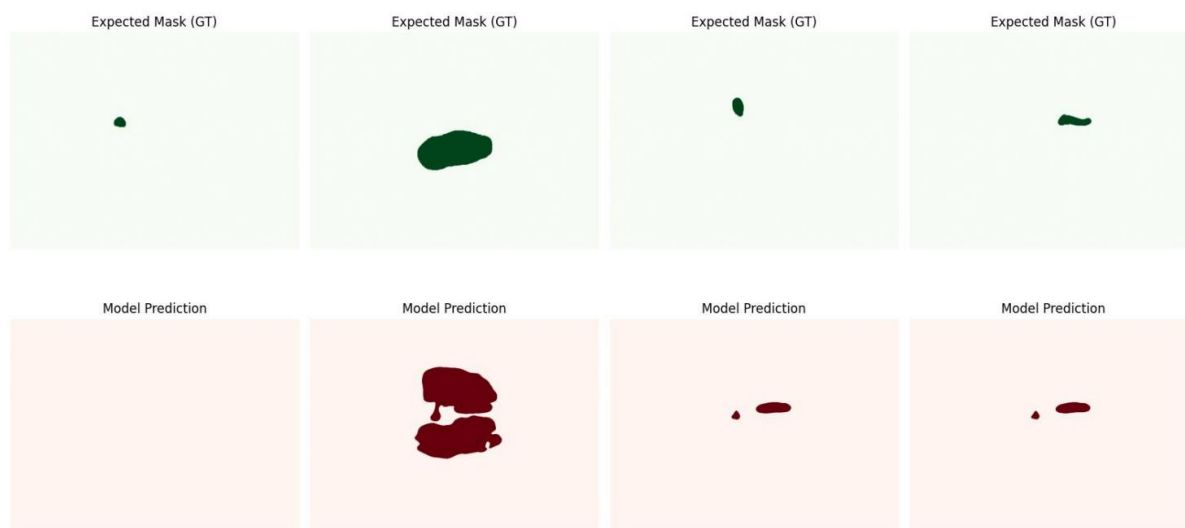
7.1 Experimentul 1 - Antrenare pe imagini brute (baseline)

Primul experiment folosește imaginile exact în forma în care se găsesc în dataset, fără niciun fel de preprocesare suplimentară. Scopul este stabilirea unui punct de referință față de care să fie evaluate modificările din experimentele ulterioare.

Rezultatele obținute pe setul de test după 90 de epoci arată un comportament nesurprinzător pentru o configurație baseline. Artera aortică nu este detectată deloc, cu un Dice score de 0.0000 și o sensibilitate de 0.0000, în ciuda specificității perfecte de 1.0000. Aceasta înseamnă că modelul prezice fundal pentru toți pixelii arterei, ceea ce îi permite să evite complet false positives dar ratează toate adevăratele pozitive. Ficatul obține cel mai bun Dice score din acest experiment, 0.5665, cu o sensibilitate ridicată de 0.8093, dar o precizie slabă de 0.4483, indicând supraestimare a suprafeței. Stomacul și vena obțin rezultate modeste, cu Dice 0.3282 și respectiv 0.4888. Macro average Dice după 90 de epoci este 0.3459.

La 150 de epoci, rezultatele se îmbunătățesc marginal. Artera rămâne nedetectată. Ficatul crește ușor la 0.5739, stomacul înregistrează cel mai mare salt, de la 0.3282 la 0.4825, iar vena scade ușor la 0.4646. Macro average ajunge la 0.3802.

Prezența textului suprapus din marginile imaginilor, generat de echipamentul ecografic, reprezintă principala sursă de zgomot în acest experiment. Modelul este antrenat pe imagini care conțin aceste artefacte și este evaluat pe imagini similare, deci consistența e păstrată, însă textul introduce pixeli cu valori extreme care pot interfera cu detectarea structurilor mici precum artera.



Figură 7.1 Comparăție Ground-truth vs. Predicția modelului la 150 de epoci ale exp. 1

7.2 Experimentul 2 - Antrenare pe imagini fără text suprapus

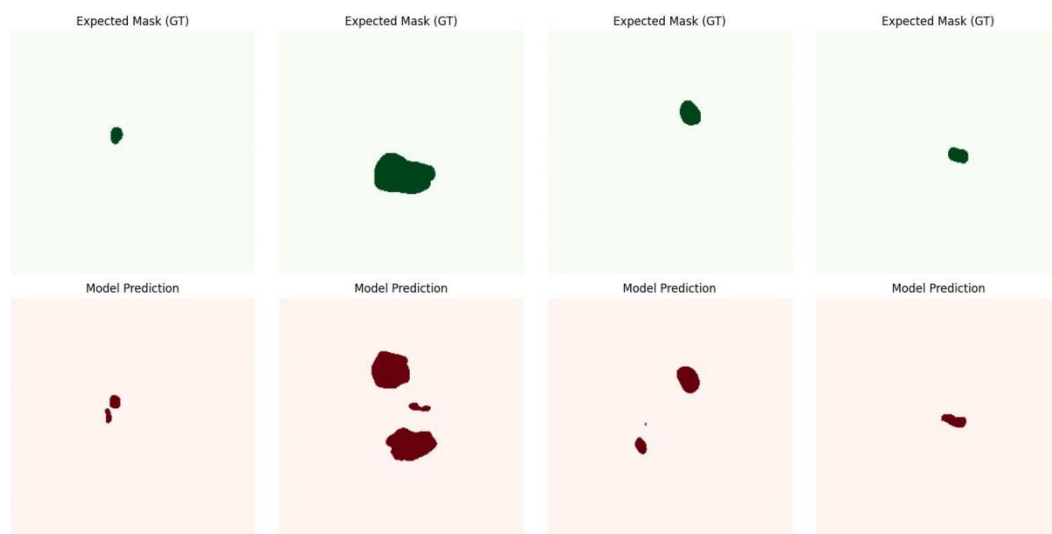
Al doilea experiment introduce eliminarea textului suprapus din imaginile ecografice înainte de antrenare și evaluare. Restul configurației rămâne identic cu Experimentul 1.

Textul suprapus de echipamentul ecografic apare în imaginile din dataset ca pixeli cu valori semnificativ diferite față de țesutul înconjurător: caractere albe pe fundal negru, cu contraste locale ridicate. Algoritmul de eliminare se bazează pe această proprietate. Pentru fiecare pixel, se calculează o medie locală a vecinătății cu radius 10, folosind un summed area table pentru eficiență. Pixelii care deviază față de această medie cu mai mult de 22 de unități în oricare direcție sunt marcați ca text. Masca rezultată este curățată de obiecte mici prin eroziune morfologică cu radius 2 urmată de dilatare cu același radius, eliminând detectările false pe texturi naturale ale țesutului. Masca finală este extinsă cu radius 3 pentru a acoperi complet marginile caracterelor. Pixelii marcați sunt înlocuiți cu valorile mediei locale calculate cu un radius mai mare de 12, producând o tranziție naturală cu fundalul imaginii fără a introduce discontinuități vizibile.

Impactul acestei modificări este imediat vizibil în rezultate. La 90 de epoci, artera aortică este detectată pentru prima dată, cu un Dice score de 0.4943 și o sensibilitate de 0.7241, față de 0.0000 în Experimentul 1. Această schimbare singulară demonstrează că textul suprapus interferează direct cu detectarea structurii cele mai mici din dataset: pixelii de text din marginile imaginii aveau valori similare cu cei ai arterei, introducând confuzie în procesul de antrenare. Ficatul crește de la 0.5665 la 0.6073, stomacul de la 0.3282 la 0.5425, iar vena de la 0.4888 la 0.6469. Macro average sare de la 0.3459 la 0.5727, o îmbunătățire de peste 65% față de baseline.

La 150 de epoci, toate structurile continuă să se îmbunătățească față de 90 de epoci. Artera ajunge la 0.5634, ficatul la 0.6339, stomacul la 0.5861 și vena la 0.6852. Macro average atinge 0.6172. Comparativ cu Experimentul 1 la același număr de epoci, îmbunătățirea relativă este de 62.3%.

Rezultatele confirmă că textul suprapus reprezenta o sursă semnificativă de zgomot pentru modelul din Experimentul 1 și că eliminarea lui are un efect pozitiv consistent pe toate structurile, nu doar pe arteră.



Figură 7.2 Comparăție Ground-truth vs. Predicția modelului la 150 de epoci ale exp. 2

7.3 Experimentul 3 - Antrenare cu ponderi ajustate per organ

Al treilea experiment păstrează eliminarea textului din Experimentul 2 și adaugă ajustarea ponderilor pozitive din funcția de pierdere BCE per organ. În configurația anterioară, toate organele aveau aceeași pondere de 25.0. În acest experiment, ponderile sunt diferențiate: 150.0 pentru arteră, 15.0 pentru ficat, 20.0 pentru stomac și 30.0 pentru venă. Implementarea folosește un tensor de shape (1, 4, 1, 1) pentru a se propaga corect peste tensorii de dimensiune (B, C, H, W):

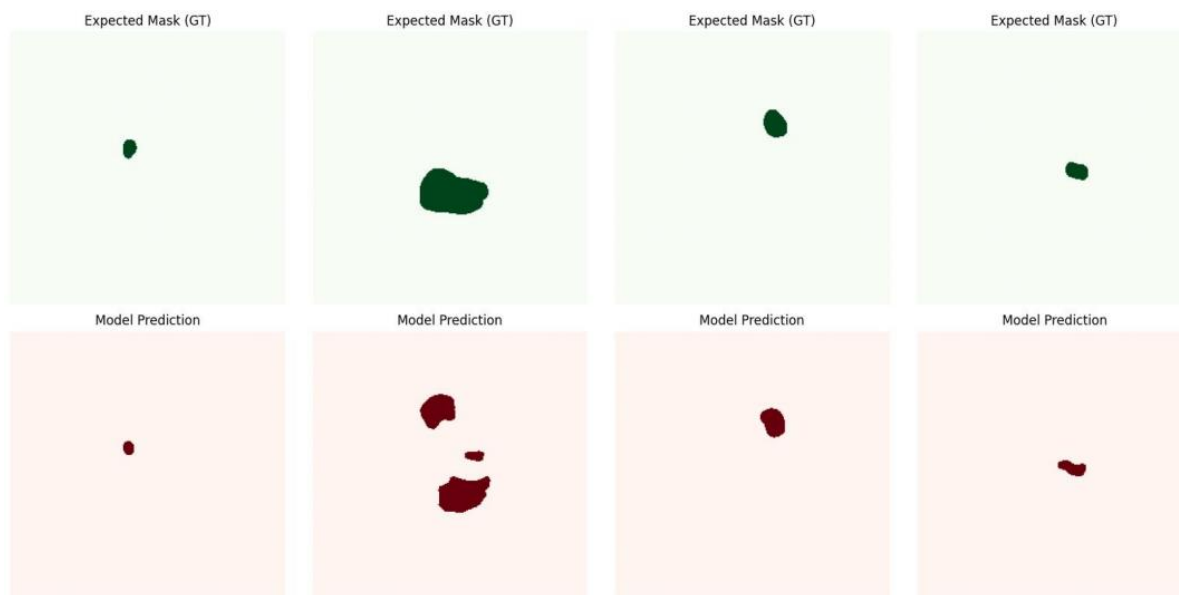
```
pos_weights = torch.tensor([150.0, 15.0, 20.0, 30.0]).view(1, -1, 1, 1).to(Config.DEVICE)
```

Decizia de a schimba ponderile a venit din observarea comportamentului modelului în timpul antrenării Experimentului 2. Deși rezultatele finale pe setul de test arătau o arteră detectată, un număr mare de epoci din procesul de validare aveau artera complet nedetectată, cu Dice 0.0000 pe setul de validare. Modelul oscila între a detecta și a nu detecta artera, fără a stabili o predicție consistentă. Ponderea ridicată de 150.0 aplicată arterei forțează rețeaua să penalizeze mult mai dur erorile pe această structură, eliminând tendința de a ignora artera în favoarea structurilor mai mari.

Rezultatele la 90 de epoci sunt surprinzătoare față de așteptări. Macro average scade la 0.2983, sub Experimentul 1 Raw. Artera coboară de la 0.4943 la 0.0994, ficatul de la 0.6073 la 0.4727, stomacul de la 0.5425 la 0.1877, iar vena de la 0.6469 la 0.4332. Explicația este că ponderea ridicată de 150.0 pentru arteră destabilizează antrenarea în fazele timpurii. Gradienții produși de BCE ponderat sunt mult mai mari decât în configurația anterioară, iar rețeaua are nevoie de mai multe epoci pentru a se adapta și a găsi un nou echilibru.

La 150 de epoci, situația se inversează complet. Macro average ajunge la 0.6302, depășind Experimentul 2 la același număr de epoci (0.6172). Artera sare la 0.5827, ficatul la 0.6476, stomacul la 0.6033 și vena la 0.6872. Comparativ cu Experimentul 2 la 150 de epoci, toate structurile se îmbunătățesc, confirmând că ponderile ajustate sunt benefice dar necesită un număr mai mare de epoci pentru convergență.

Această dinamică ilustrează un compromis important: ponderile ridicate per organ îmbunătățesc rezultatele finale dar îngreunează convergența timpurie. La 90 de epoci, Experimentul 3 este cel mai slab dintre toate; la 150 de epoci, cel mai bun de până acum



Figură 7.3 Comparație Ground-truth vs. Predicția modelului la 150 de epoci ale exp. 3

7.4 Experimentul 4 - Antrenare cu filtru CLAHE

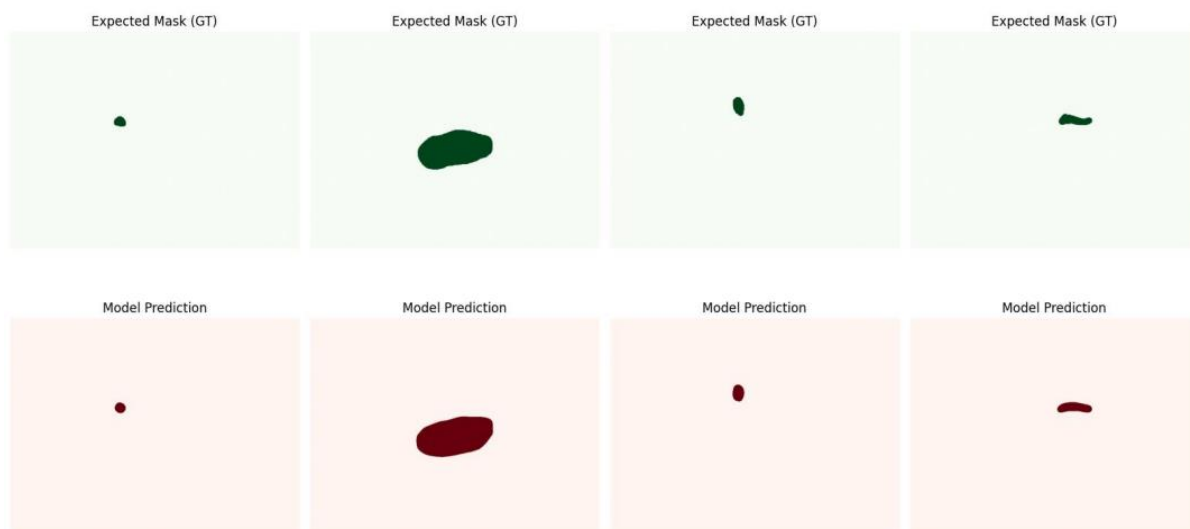
Al patrulea experiment adaugă filtrul CLAHE peste configurația Experimentului 3: eliminare text, ponderi ajustate per organ și, în plus, îmbunătățire adaptivă a contrastului local înainte de antrenare și evaluare. CLAHE este aplicat cu clipLimit 2.0 și tileSize 8×8, pe imaginea curățată de text. [29]

CLAHE, spre deosebire de egalizarea histogramei globale, operează local pe regiuni mici ale imaginii, limitând amplificarea contrastului la o valoare maximă controlată de clipLimit. Efectul practic pe imaginile ecografice este că structurile anatomice cu contrast scăzut față de țesutul înconjurător devin mai distincte, fără a amplifica zgomotul din zonele deja bine contrastante. Imaginile ecografice abdominale fetale au în mod natural un contrast neuniform: unele structuri sunt clar vizibile, altele abia se disting de fundal, în funcție de adâncimea de penetrare a ultrasunetelor și de caracteristicile individuale ale pacientei. [29]

Rezultatele la 90 de epoci arată îmbunătățiri semnificative față de toate experimentele anterioare pe majoritatea structurilor, dar cu un comportament neuniform. Ficatul înregistrează cel mai mare Dice score de până acum, 0.8512, cu o îmbunătățire de peste 30% față de Experimentul 3. Stomacul ajunge la 0.6540 și vena la 0.5420. Artera, în schimb, scade față de Experimentul 3 la 150 de epoci, obținând 0.3650 față de 0.5827. Macro average la 90 de epoci este 0.6030, mai mic decât Experimentul 3 la 150 de epoci, ceea ce sugerează că și acest experiment are nevoie de mai multe epoci pentru a stabili detectarea arterei.

La 150 de epoci, Experimentul 4 produce cele mai bune rezultate din toate experimentele, cu o marjă semnificativă. Artera ajunge la 0.8355, ficatul la 0.8921, stomacul la 0.8722 și vena la 0.8552. Macro average atinge 0.8637, față de 0.6302 al Experimentului 3, o îmbunătățire relativă de 37%. Test loss scade la 0.1224, cel mai mic din toate experimentele. Toate cele patru structuri depășesc pragul de 0.83 Dice, inclusiv artera, care în Experimentele 1 și 2 era fie nedetectată, fie instabilă.

Saltul de performanță față de Experimentul 3 confirmă că CLAHE aduce o contribuție esențială la calitatea segmentării, independent de ponderile din funcția de pierdere. Îmbunătățirea contrastului local face ca structurile anatomice să fie mai ușor de delimitat de rețea, reducând ambiguitatea la granițele dintre structuri și țesutul înconjurător.



Figură 7.4 Comparație Ground-truth vs. Predicția modelului la 150 de epoci ale exp. 4

7.5 Comparație globală între experimente

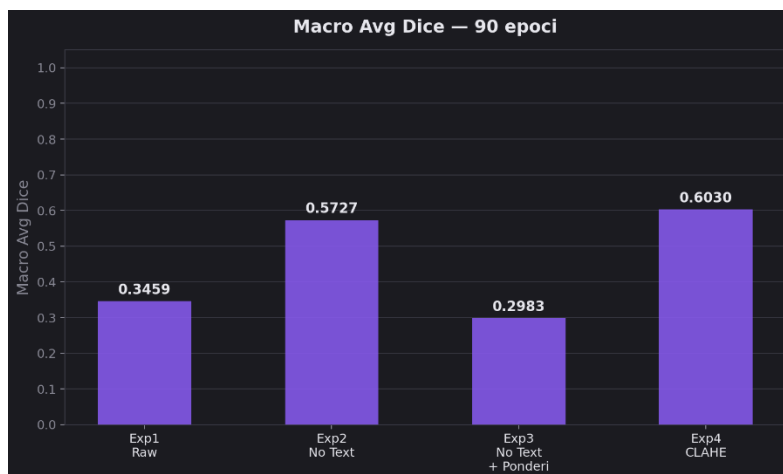
Cele patru experimente urmează o progresie incrementală, fiecare modificare adăugând o componentă nouă față de configurația anterioară. Privind rezultatele la 150 de epoci, care reprezintă configurația finală a fiecărui experiment, trendul general este de îmbunătățire continuă, cu excepția Experimentului 3 la 90 de epoci, unde convergența întârziată a produs temporar o regresie față de Experimentul 2.

Cea mai importantă îmbunătățire individuală este eliminarea textului suprapus, introdusă în Experimentul 2. Trecerea de la Exp1 la Exp2 produce cel mai mare salt relativ din întreaga serie, cu Macro Avg Dice crescând de la 0.3802 la 0.6172, o diferență de 0.2370. Efectul este cel mai vizibil pe arteră, care trece de la 0.0000 la 0.5634, confirmând că textul suprapus era principala cauză a nedectării acestei structuri în baseline.

Ajustarea ponderilor din Experimentul 3 aduce un câștig modest la 150 de epoci față de Exp2, de la 0.6172 la 0.6302. Contribuția sa nu este vizibilă în cifra finală de Macro Avg, ci în stabilitatea detectării arterei pe parcursul antrenării: numărul de epoci cu artera nedectată pe setul de validare scade semnificativ față de Experimentul 2.

Adăugarea CLAHE în Experimentul 4 produce cel mai mare salt absolut la 150 de epoci, de la 0.6302 la 0.8637, o diferență de 0.2335. Spre deosebire de modificările anterioare, îmbunătățirea este uniformă pe toate cele patru structuri: niciun organ nu scade față de Experimentul 3, iar artera, ficatul, stomacul și vena depășesc toate pragul de 0.83 Dice.

7.6 Rezultate la 90 de epoci



Figură 7.5 Evoluția din timpul antrenării la 90 de epoci a mediei metricilor

Experiment	Organ	Dice	IoU	Sens	Spec	Prec
Exp1 Raw	artery	0.0000	0.0000	0.0000	1.0000	0.0000
	liver	0.5665	0.4029	0.8093	0.9598	0.4483
	stomach	0.3282	0.2120	0.4256	0.9916	0.2841
	vein	0.4888	0.3296	0.7152	0.9939	0.3843
	Macro	0.3459				

Tabel 7.1 Rezultatele la 90 de epoci pentru experimentul 1

Experiment	Organ	Dice	IoU	Sens	Spec	Prec
Exp2 No Text	artery	0.4943	0.3477	0.7241	0.9969	0.4108
	liver	0.6073	0.445	0.8664	0.961	0.4784
	stomach	0.5425	0.4086	0.6004	0.9967	0.5246
	vein	0.6469	0.4875	0.7416	0.9974	0.5997
	Macro	0.5727				

Tabel 7.2 Rezultatele la 90 de epoci pentru experimentul 2

Experiment	Organ	Dice	IoU	Sens	Spec	Prec
Exp3 No Text + Ponderi	artery	0.0994	0.0529	0.5048	0.9868	0.0568
	liver	0.4727	0.3168	0.6506	0.9589	0.3802
	stomach	0.1877	0.1115	0.2429	0.9904	0.1757
	vein	0.4332	0.2875	0.5014	0.9952	0.4389
	Macro	0.2983				

Tabel 7.3 Rezultatele la 90 de epoci pentru experimentul 3

Experiment	Organ	Dice	IoU	Sens	Spec	Prec
Exp4 CLAHE	artery	0.365	0.2232	0.421	0.995	0.322
	liver	0.8512	0.7409	0.8805	0.9912	0.8237
	stomach	0.654	0.4858	0.712	0.9845	0.6047
	vein	0.542	0.3717	0.61	0.988	0.4876
	Macro	0.603				

Tabel 7.4 Rezultatele la 90 de epoci pentru experimentul 4

7.7 Rezultate la 150 de epoci



Figură 7.6 Evoluția din timpul antrenării la 150 de epoci a mediei metricilor

Experiment	Organ	Dice	IoU	Sens	Spec	Prec
Exp1 Raw	artery	0	0	0	1	0
	liver	0.5739	0.4033	0.8369	0.9582	0.4377
	stomach	0.4825	0.3216	0.6657	0.9932	0.3844
	vein	0.4646	0.3044	0.6969	0.993	0.3506
	Macro	0.3802				

Tabel 7.5 Rezultatele la 150 de epoci pentru experimentul 1

Experiment	Organ	Dice	IoU	Sens	Spec	Prec
Exp2 No Text	artery	0.5634	0.4187	0.6835	0.9985	0.5156
	liver	0.6339	0.4739	0.8705	0.9653	0.5118
	stomach	0.5861	0.45	0.6497	0.9966	0.5781
	vein	0.6852	0.5289	0.758	0.9978	0.6451
	Macro	0.6172				

Tabel 7.6 Rezultatele la 150 de epoci pentru experimentul 2

Experiment	Organ	Dice	IoU	Sens	Spec	Prec
Exp3 No Text + Ponderi	artery	0.5827	0.4405	0.6715	0.9976	0.5623
	liver	0.6476	0.4902	0.8142	0.9723	0.5546
	stomach	0.6033	0.4673	0.6525	0.9969	0.6113
	vein	0.6872	0.5339	0.7387	0.9976	0.6791
	Macro	0.6302				

Tabel 7.7 Rezultatele la 150 de epoci pentru experimentul 3

Experiment	Organ	Dice	IoU	Sens	Spec	Prec
Exp4 CLAHE	artery	0.8355	0.7219	0.8642	0.9997	0.8113
	liver	0.8921	0.8055	0.969	0.9921	0.827
	stomach	0.8722	0.7755	0.9136	0.9989	0.8357
	vein	0.8552	0.7475	0.901	0.9989	0.8148
	Macro	0.8637				

Tabel 7.8 Rezultatele la 150 de epoci pentru experimentul 4

7.8 Analiză vizuală a predicțiilor

Analiza vizuală completează metricile numerice și oferă o perspectivă directă asupra comportamentului modelului pe cazuri individuale din setul de test.

În Experimentul 1, predicțiile reflectă comportamentul numeric descris anterior. Artera nu este prezisă deloc, masca corespunzătoare rămânând complet neagră. Ficatul este detectat, dar masca prezisă este fragmentată și supraestimată față de ground truth: modelul prezice o suprafață mai mare decât cea reală, ceea ce explică sensibilitatea ridicată (0.8093) combinată cu precizia slabă (0.4483). Stomacul și vena sunt detectate parțial, cu măști mici și imprecise față de contururile ground truth.

În Experimentul 2, diferența față de baseline este vizibilă imediat pe masca arterei, care apare pentru prima dată ca o predicție pozitivă, chiar dacă dimensiunea ei este mai mică decât ground truth. Ficatul rămâne supraestimat, dar fragmentarea dispare. Stomacul și vena au măști mai apropiate de ground truth față de Exp1.

În Experimentul 3 la 150 de epoci, predicțiile sunt mai curate față de Exp2. Artera are o mască mai compactă și mai apropiată de dimensiunea ground truth. Ficatul și stomacul prezintă contururi mai precise, iar vena produce cea mai bună predicție de până acum, cu o formă alungită care urmărește mai fidel structura reală.

Experimentul 4 produce vizual cea mai clară diferență față de toate experimentele anterioare. Măștile pentru toate cele patru structuri sunt compacte, bine delimitate și apropiate de ground truth. Artera, care în Exp1 era absentă și în Exp2-3 era instabilă, apare acum ca o predicție consistentă cu dimensiune și poziție corecte. Ficatul nu mai este supraestimat: masca prezisă urmărește fidel conturul ground truth. Stomacul și vena au predicții aproape superpozabile cu ground truth pe exemplele din setul de test.

Un pattern vizibil în toate experimentele este că modelul tinde să producă măști mai mici decât ground truth pe structurile mici, în special pe arteră și venă. Aceasta este consecința directă a dezechilibrului de scară: chiar și cu ponderi ajustate, rețeaua preferă să fie conservatoare pe structurile mici pentru a evita fals-positivle. La Exp4, acest comportament este cel mai redus, dar rămâne prezent pe unele exemple din setul de test.

7.9 Concluzie generală a experimentelor

Seria de experimente confirmă că performanța unui model de segmentare depinde mai mult de calitatea datelor de intrare decât de modificările arhitecturale sau ale funcției de pierdere. Fiecare experiment a adăugat o singură modificare față de cel anterior, iar efectul fiecăreia a putut fi izolat și măsurat direct. Eliminarea textului suprapus a produs cel mai mare salt relativ, demonstrând că artefactele de achiziție pot bloca complet detectarea structurilor mici. Ajustarea ponderilor per organ nu a produs un câștig numeric imediat, ci a stabilizat comportamentul modelului pe parcursul antrenării, reducând oscilațiile în detectarea arterei. Adăugarea CLAHE a produs cel mai mare câștig absolut și singurul care a fost uniform pe toate structurile simultan. La 150 de epoci, configurația finală atinge un Macro Avg Dice de 0.8637, față de 0.3802 în baseline, o îmbunătățire de 127% obținută exclusiv prin modificări de preprocesare și configurare a funcției de pierdere, fără nicio schimbare a arhitecturii rețelei.

8 Aplicație demo pentru inferență interactivă

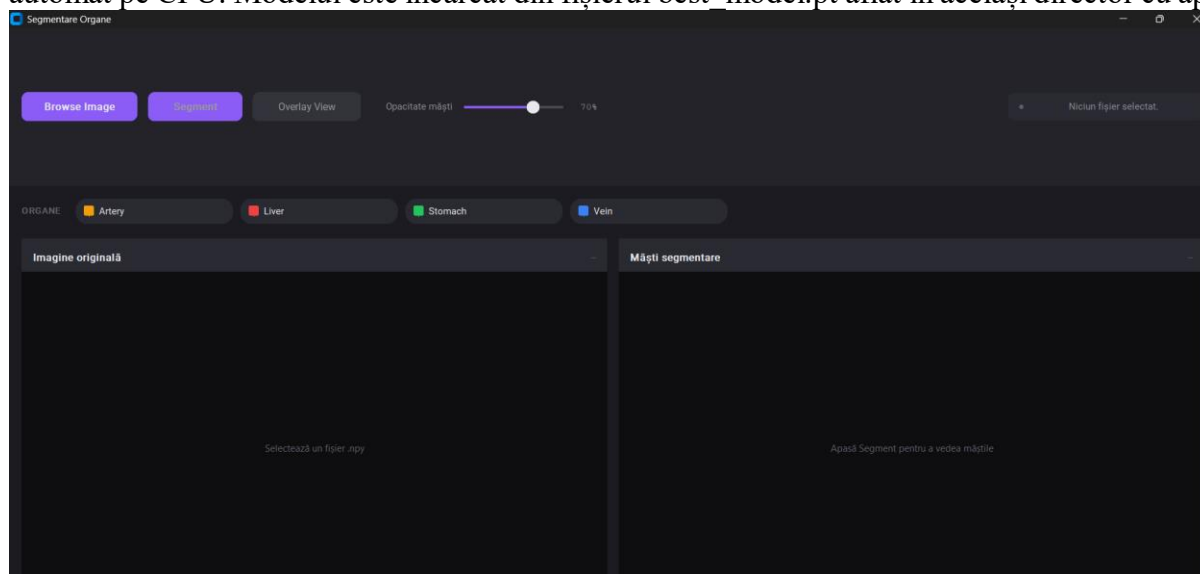
8.1 Prezentare generală

Pentru a facilita evaluarea vizuală a modelului antrenat, a fost dezvoltată o aplicație desktop de inferență interactivă. Scopul ei este să permită aplicarea modelului pe imagini din dataset fără a necesita acces la mediul de antrenare sau cunoștințe de programare. Utilizatorul selectează un fișier din dataset, apasă un buton și vede imediat măștile de segmentare produse de model.

Aplicația este construită în Python, folosind biblioteca CustomTkinter pentru interfața grafică, cu tema dark. Ferestrele și panourile sunt gestionate nativ prin Tkinter, iar imaginile sunt afișate prin matplotlib integrat direct în interfață prin FigureCanvasTkAgg, fără ferestre externe. Inferența folosește același model MultiOutputUNet și același pipeline de preprocesare ca Experimentul 4: eliminarea textului suprapus urmată de CLAHE.

Aplicația acceptă la intrare fișiere în formatul NPY al dataset-ului, extrage imaginea din dicționarul stocat și o procesează automat înainte de inferență. Rezultatele sunt afișate în două forme: o grilă cu cele patru măști binare și un overlay color suprapus pe imaginea originală. Fiecare structură are o culoare fixă: galben-amber pentru arteră, roșu pentru ficat, verde pentru stomac și albastru pentru venă.

Aplicația rulează local pe Windows și folosește GPU dacă acesta este disponibil, altfel comută automat pe CPU. Modelul este încărcat din fișierul `best_model.pt` aflat în același director cu aplicația.



Figură 8.1 Ecranul principal al aplicației

8.2 Arhitectura aplicației și tehnologii folosite

Aplicația este structurată în trei componente principale: interfața grafică, pipeline-ul de preprocesare și modulul de inferență. Toate trei rulează în același proces Python, cu inferența delegată pe un thread separat pentru a nu bloca interfața.

Interfața grafică este construită cu CustomTkinter, o extensie a bibliotecii standard Tkinter care adaugă stilizare modernă și suport pentru tema dark. CustomTkinter a fost ales față de alte soluții precum PyQt sau wxPython pentru că nu necesită instalare de dependențe externe complexe și produce o interfață nativă pe Windows fără configurare suplimentară. Afișarea imaginilor ecografice și a măștilor de segmentare este gestionată de matplotlib, integrat direct în fereastra aplicației prin

componenta `FigureCanvasTkAgg`. Această abordare permite redarea imaginilor grayscale și a overlay-urilor RGBA în același viewport cu interfața, fără ferestre `matplotlib` separate.

Preprocesarea imaginilor folosește `NumPy` pentru operațiile morfologice de curățare a textului și `OpenCV` pentru aplicarea filtrului CLAHE. Normalizarea și conversia la tensor `PyTorch` sunt realizate cu `Albumentations` [25], același transform folosit la evaluarea modelului în antrenare.

Inferența folosește `PyTorch` cu detecție automată a device-ului disponibil:

```
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
```

Modelul este încărcat o singură dată la pornirea aplicației și refolosit pentru toate inferențele ulterioare din sesiune. Suportă două formate de checkpoint: `state_dict` direct sau dicționar cu cheia `model_state_dict`, pentru compatibilitate cu ambele variante de salvare folosite în antrenare.

Inferența rulează pe un thread daemon separat față de thread-ul principal al interfeței:

```
threading.Thread(target=worker, daemon=True).start()
```

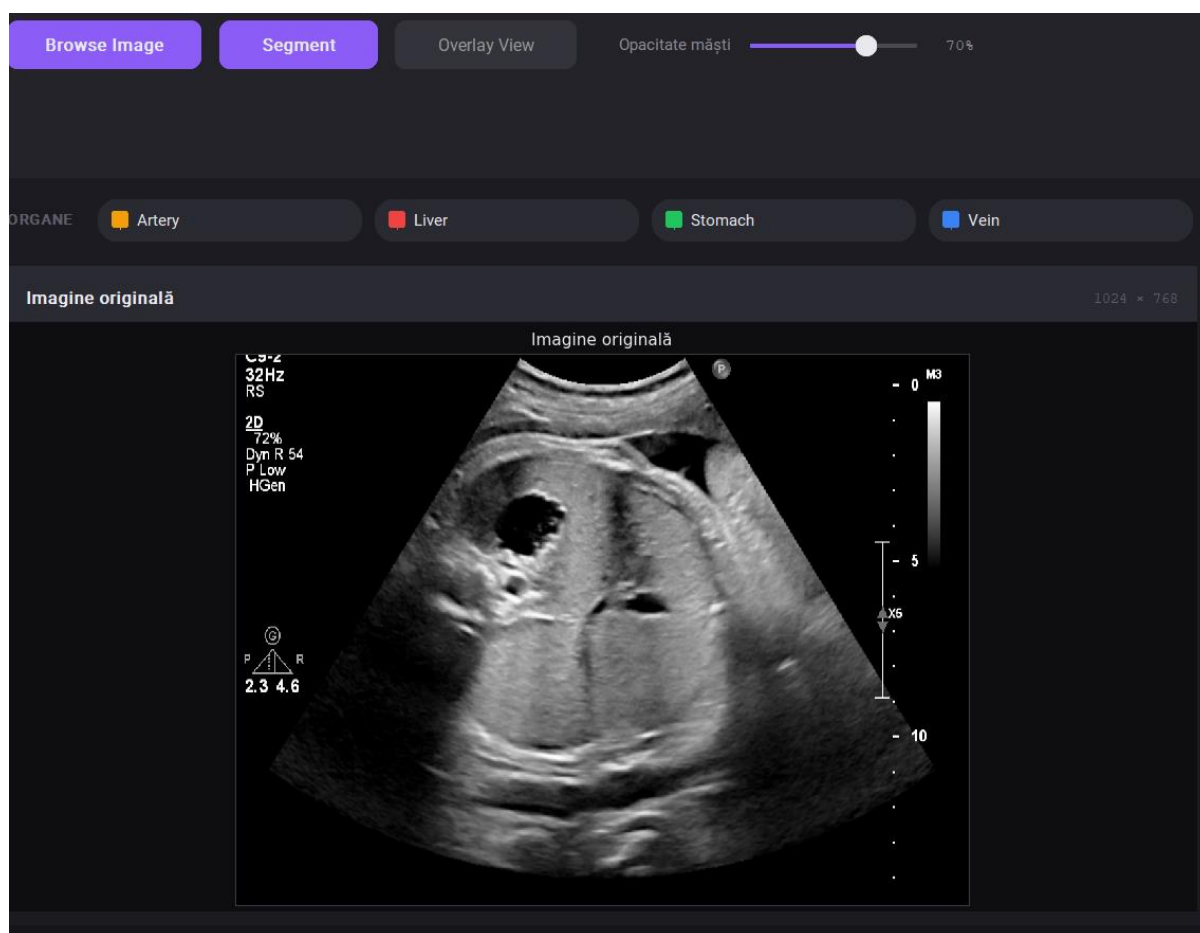
Această separare este necesară deoarece inferența pe CPU poate dura câteva secunde, timp în care interfața ar îngheța dacă totul ar rula pe același thread. Rezultatele sunt trimise înapoi pe thread-ul principal prin `self.after(0, callback)`, mecanismul standard Tkinter pentru comunicare inter-thread.

8.3 Interfața grafică

Fereastra principală are rezoluția implicită de 1280×820 pixeli și este împărțită în cinci zone distincte: bara de stare, toolbar-ul, bara de organe, panourile de vizualizare și bara de status.

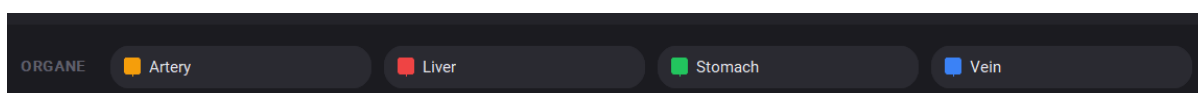
Bara de stare este plasată în partea de jos a ferestrei și afișează în permanență numele fișierului curent cu dimensiunile sale, numele fișierului model încărcat și tema activă. Este actualizată automat la fiecare acțiune a utilizatorului.

Toolbar-ul ocupă partea de sus a ferestrei și conține toate comenzile principale. Butonul `Browse Image` deschide un dialog de selectare a fișierelor, filtrat exclusiv pentru fișiere NPY. Butonul `Segment` pornește inferența și este dezactivat până la încărcarea unei imagini. Butonul `Overlay View` activează suprapunerea măștilor pe imaginea originală și este dezactivat până după prima inferență. Glisorul de opacitate controlează transparența măștilor în modul overlay, cu valori între 0 și 100%, implicit 70%. În dreapta toolbar-ului se află o etichetă rotunjită de stare care afișează mesaje contextuale: "Niciun fișier selectat", numele fișierului încărcat sau numărul de organe segmentate după inferență.



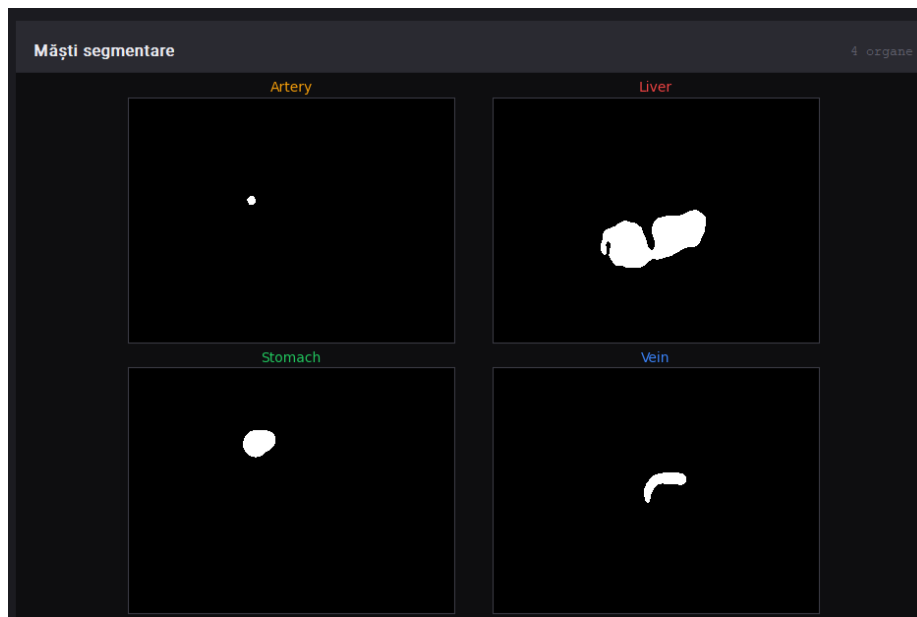
Figură 8.2 Aplicația după upload de imagine

Bara de organe este plasată imediat sub toolbar și conține patru butoane interactive, câte unul per structură. Fiecare buton are o mostră de culoare și numele organului. Click pe un chip activează vizibilitatea organului respectiv în ambele panouri simultan. Când un organ este dezactivat, chip-ul devine estompat, masca lui dispare din panoul drept și culoarea lui dispare din overlay. Culoarea fixă sunt: galben-amber pentru arteră, roșu pentru ficat, verde pentru stomac și albastru pentru venă.



Figură 8.3 Toolbar-ul / Legenda

Panourile de vizualizare ocupă cea mai mare parte a ferestrei și sunt împărțite în două coloane egale. Panoul stâng, intitulat "Imagine originală", afișează imaginea ecografică brută după încărcare. Când overlay-ul este activ, afișează aceeași imagine cu măștile de segmentare suprapuse color la opacitatea setată, împreună cu o legendă în colțul din dreapta sus. Panoul drept, intitulat "Măști segmentare", afișează o grilă 2x2 cu cele patru măști binare produse de model, fiecare cu titlul colorat în culoarea organului corespunzător. Pe durata inferenței, panoul drept este înlocuit cu o bară de progres indeterminată și textul "Segmentare în curs...".



Figură 8.4 Măștile de segmentare

8.4 Pipeline-ul de preprocesare și inferență

Când utilizatorul apasă butonul Segment, aplicația execută automat un pipeline de preprocesare înainte de a transmite imaginea modelului. Același pipeline a fost folosit în Experimentul 4, deci rezultatele produse de aplicație sunt direct comparabile cu rezultatele raportate în capitolul de experimente.

Prima etapă este eliminarea textului suprapus de echipamentul de achiziție. Imaginile din dataset conțin text și simboluri în marginile stânga și dreapta, generate automat de aparatul ecografic. Acestea nu fac parte din anatomia fetală și pot introduce artefacte în predicțiile modelului. Detecția se bazează pe o filtrare medie locală cu radius 10, calculată eficient prin summed area table:

```
def clean_text(image):
    local_mean = mean_filter(image, radius=10)
    text_mask = (image >= local_mean + 22) | (image <= local_mean - 22)
    text_mask = remove_small_objects(text_mask, radius=2)
    text_mask = binary_dilate(text_mask, radius=3)
    background = mean_filter(image, radius=12).astype(np.uint8)
    clean = image.copy()
    clean[text_mask] = background[text_mask]
    return clean
```

Pixelii care deviază cu mai mult de 22 față de media locală sunt marcați ca text. Maska rezultată este curățată de obiecte mici prin eroziune morfologică urmată de dilatare cu radius 2, apoi extinsă cu radius 3 pentru a acoperi complet marginile caracterelor. Pixelii detectați ca text sunt înlocuiți cu valorile mediei locale calculate cu radius 12, producând o tranziție naturală cu fundalul imaginii.

A doua etapă este aplicarea filtrului CLAHE cu clipLimit 2.0 și tileGridSize 8×8:

```
def preprocess_for_inference(image):
```



```
clean = clean_text(image)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
return clahe.apply(clean)
```

CLAHE îmbunătățește contrastul local al imaginii ecografice, făcând structurile anatomice mai distincte fără a amplifica zgomotul global.

După preprocesare, imaginea este normalizată cu media 0.5 și deviația standard 0.5, convertită la tensor PyTorch și transmisă modelului. Inferența rulează cu `torch.no_grad()` pentru a dezactiva calculul gradientilor, reducând consumul de memorie. Sigmoid aplicat pe logits-urile modelului urmată de pragul 0.5 produce cele patru măști binare:

```
def run_inference(model, image_2d, device):
    augmented = val_transform(image=image_2d)
    tensor = augmented['image'].unsqueeze(0).to(device)
    with torch.no_grad():
        logits = model(tensor)
        preds = (torch.sigmoid(logits) > 0.5).float()
    return preds[0].cpu().numpy()
```

Rezultatul este un array NumPy de shape (4, H, W) cu valori binare, transmis direct modulului de vizualizare.

8.5 Vizualizarea rezultatelor

După finalizarea inferenței, rezultatele sunt afișate simultan în ambele panouri ale interfeței.

Panoul drept afișează o grilă 2×2 cu cele patru măști binare de segmentare, câte una per structură. Fiecare subparcelă are titlul colorat în culoarea organului corespunzător: galben-amber pentru arteră, roșu pentru ficat, verde pentru stomac și albastru pentru venă. Măștile sunt afișate în grayscale, cu pixelii de foreground albi și fundalul negru. Dacă un organ nu a fost detectat în imaginea respectivă, masca corespunzătoare rămâne complet neagră.

Panoul stâng oferă modul overlay, activat prin butonul Overlay View din toolbar. În acest mod, imaginea ecografică originală este afișată în grayscale, cu măștile de segmentare suprapuse ca straturi RGBA semitransparente. Fiecare organ are culoarea sa fixă, iar transparența este controlată prin glisorul de opacitate. La valoarea implicită de 70%, structurile anatomice din imaginea originală rămân vizibile prin măști, permițând evaluarea vizuală a acurateții segmentării. O legendă cu culorile organelor apare automat în colțul din dreapta sus al imaginii când overlay-ul este activ.

Implementarea overlay-ului construiește pentru fiecare organ un array RGBA în care canalul alpha este produsul dintre masca binară și opacitatea curentă:

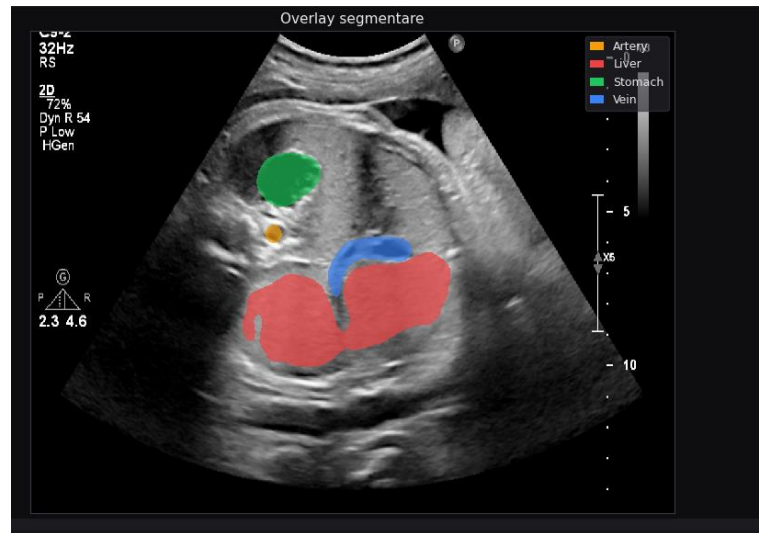
```
rgba = np.zeros((*mask.shape, 4), dtype=np.float32)
rgba[..., 0] = r
rgba[..., 1] = g
```

```

rgba[..., 2] = b
rgba[..., 3] = mask * self._opacity
ax.imshow(rgba, interpolation='nearest')

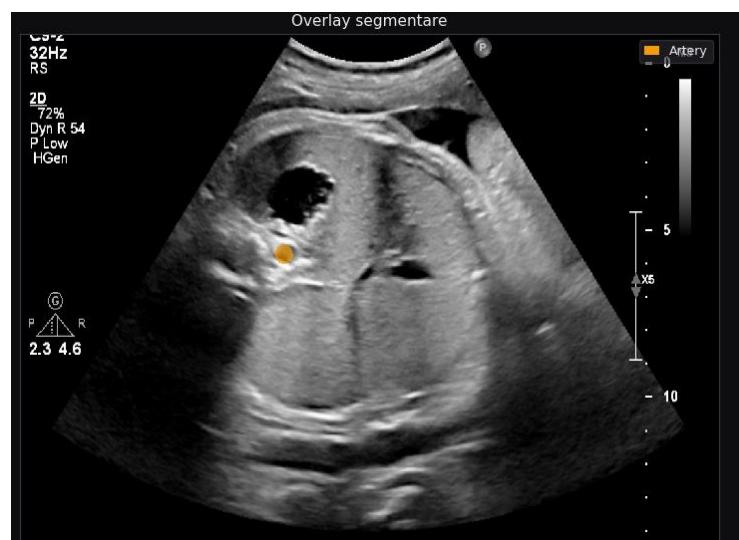
```

Organele sunt afișate în ordinea: arteră, ficat, stomac, venă. Dacă două structuri se suprapun în predicție, cea afișată ultima acoperă vizual celelalte.



Figură 8.5 Overlay-ul de organe pe imaginea originală

Vizibilitatea fiecărui organ poate fi controlată independent prin butoane din bara de organe. Dezactivarea unui organ îl elimină simultan din grila de măști și din overlay, fără a fi necesară o nouă inferență. Această funcționalitate permite analiza izolată a fiecărei structuri, utilă în special pentru artera aortică, a cărei mască este mult mai mică decât celelalte și poate fi greu de observat când toate organele sunt active simultan.



Figură 8.6 Overlay doar cu artera aortică

8.6 Gestionarea erorilor și aspecte de implementare

La pornirea aplicației, primul lucru verificat este existența fișierului `best_model.pt` în același director cu `app.py`. Dacă fișierul lipsește, utilizatorul primește un mesaj de eroare explicit cu calea așteptată și instrucțiunea de remediere, iar aplicația rămâne deschisă fără model încărcat. Butoanele de inferență rămân dezactivate până la rezolvarea problemei și repornirea aplicației.

La încărcarea unui fișier NPY, aplicația verifică că fișierul conține structura de dicționar așteptată cu cheia `image` și că imaginea extrasă este bidimensională după procesare. Dacă oricare dintre aceste condiții nu este îndeplinită, utilizatorul primește un mesaj de eroare cu detalii despre problema detectată, iar starea aplicației rămâne neschimbată.

Încărcarea modelului suportă două formate de checkpoint, pentru compatibilitate cu ambele variante produse în antrenare:

```
checkpoint = torch.load(MODEL_PATH, map_location=device)
if isinstance(checkpoint, dict) and 'model_state_dict' in checkpoint:
    checkpoint = checkpoint['model_state_dict']
model.load_state_dict(checkpoint)
```

Dacă `torch.load` eșuează din cauza incompatibilității între versiuni de NumPy sau PyTorch, eroarea este capturată și afișată utilizatorului.

Inferența rulează pe un thread daemon separat, ceea ce înseamnă că thread-ul este oprit automat când fereastra principală este închisă, fără a necesita gestionare explicită. Pe durata inferenței, butoanele `Browse` și `Segment` sunt dezactivate pentru a preveni lansarea unei a doua inferențe în paralel sau încărcarea unui fișier nou în timp ce modelul procesează imaginea curentă. După finalizare, indiferent de succes sau eroare, butoanele sunt reactivate și interfața revine la starea normală.

Redesenarea imaginilor în panouri este separat în două funcții independente, `_redraw_left` și `_redraw_right`, apelate selectiv în funcție de acțiunea utilizatorului. Modificarea opacității sau activarea unui organ nu declanșează re-inferența, ci doar redesenarea vizuală a rezultatelor deja calculate. Acest lucru face ca interacțiunile de vizualizare să fie instantanee, indiferent de dimensiunea imaginii sau de device-ul disponibil.

Concluzii și direcții viitoare

Concluzii

Lucrarea de față a abordat problema segmentării automate a patru structuri anatomice fetale din imagini ecografice abdominale 2D: artera aortică abdominală, vena ombilicală intrahepatică, ficatul fetal și stomacul fetal. Arhitectura propusă, MultiOutputUNet, combină encoderul ResNet50 pre-antrenat cu structura decoder-encoder a U-Net și tratează problema ca o sarcină de segmentare multi-label, producând simultan patru măști binare dintr-o singură trecere înainte prin rețea.

Obiectivul principal al lucrării a fost atins. Experimentul final obține un Macro Avg Dice de 0.8637 pe setul de test, cu toate cele patru structuri depășind pragul de 0.83. Artera aortică, cea mai mică structură din dataset și cea mai dificil de detectat, ajunge la un Dice score de 0.8355, față de 0.0000 în experimentul baseline. Acest progres nu a venit dintr-o singură modificare, ci dintr-o serie de decizii incrementale, fiecare justificată de rezultatele experimentului anterior.

Seria de experimente a arătat că preprocesarea imaginilor are un impact mai mare asupra performanței finale decât ajustările arhitecturale sau ale funcției de pierdere. Eliminarea textului suprapus a produs cel mai mare salt față de baseline. Ajustarea ponderilor per organ a stabilizat detectarea arterei pe parcursul antrenării. Adăugarea CLAHE a dus la cel mai mare câștig absolut, uniformizând performanța pe toate structurile simultan.

O limitare importantă a acestei lucrări este dimensiunea redusă a dataset-ului, cu 169 de pacienți și 1.588 de imagini. Un set de date mai mare ar permite explorarea unor arhitecturi mai complexe și a unor strategii de augmentare mai agresive. O altă limitare este numărul redus de experimente efectuate, impus de costul computațional al antrenării pe Google Colab A100 și de timpul limitat disponibil. Fiecare rulare completă de 150 de epoci durează între 5 și 7 ore, ceea ce a făcut imposibilă explorarea exhaustivă a spațiului de hiperparametri.

Contribuții personale

Contribuțiile proprii ale acestei lucrări sunt următoarele. Algoritmul de eliminare a textului suprapus din imaginile ecografice a fost proiectat și implementat de la zero, fără a folosi biblioteci specializate de procesare a textului în imagini. Decizia de a ajusta ponderile BCE per organ și valorile concrete ale acestora au rezultat din analiza comportamentului modelului în timpul antrenării, observând că artera era sistematic ignorată în epocile de validare. Decizia de a aplica CLAHE ca etapă finală de preprocesare a venit din analiza vizuală a imaginilor brute și a celor curățate de text, unde structurile anatomice rămăneau insuficient de contrastante. Aplicația desktop de inferență interactivă a fost implementată integral, incluzând interfața grafică, pipeline-ul de preprocesare și mecanismul de vizualizare cu overlay controlabil per organ. Interpretarea rezultatelor și deciziile de trecere de la un experiment la altul reprezintă contribuții de analiză proprie, nepreluate din literatură.

Direcții viitoare

Mai multe direcții de îmbunătățire rămân neexplorate din cauza constrângerilor computaționale. O direcție evidentă este extinderea dataset-ului, fie prin colectarea de imagini suplimentare, fie prin augmentare mai agresivă, care ar putea include umplerea imaginilor la dimensiunea completă a ecranului și eliminarea bordurii negre. Explorarea altor strategii de preprocesare, cum ar fi filtre alternative față de CLAHE sau eliminarea zgomotului gaussian, ar putea

aduce câștiguri suplimentare fără modificări arhitecturale. Ajustarea mai fină a ponderilor per organ, prin căutare sistematică în loc de valori fixe, ar putea îmbunătăți în special detectarea arterei. Post-procesarea măștilor produse de model, prin eliminarea componentelor conexe mici sau prin aplicarea de operații morfologice la inferență, ar putea reduce fals-positivile izolate vizibile în unele predicții. O extensie mai ambițioasă ar fi adaptarea soluției pentru procesarea secvențelor video ecografice, unde continuitatea temporală între cadre ar putea îmbunătăți consistența predicțiilor. Ținta de Dice score de 0.95 pe toate structurile rămâne realizabilă cu un dataset mai mare și mai mult timp de experimentare.

Bibliografie

- [1] <https://www.medlife.ro/articole-medicale/ecografia-de-sarcina-trimestrul-1-2-3-cand-este-recomandata-si-cum-te-pregatesti> accesat la data 13/05/2026
- [2] <https://novogyn.ro/obstetrica-ginecologie/rolul-ecografiei-3d-4d-evaluarea-antenatala/> accesat la data 13/05/2026
- [3] <https://www.kenhub.com/en/library/anatomy/umbilical-vein> accesat la data 21/05/2026
- [4] <https://www.verywellhealth.com/abdominal-aorta-anatomy-4587592> accesat la data 08/06/2026
- [5] Lotto, J., Stephan, T.L. & Hoodless, P.A. Fetal liver development and implications for liver disease pathogenesis. *Nat Rev Gastroenterol Hepatol* **20**, 561–581 (2023).
- [6] Noble JA, Boukerroui D. Ultrasound image segmentation: a survey. *IEEE Trans Med Imaging*. 2006 Aug;25(8):987-1010. doi: 10.1109/tmi.2006.877092. PMID: 16894993.
- [7] Litjens G, Kooi T, Bejnordi BE, Setio AAA, Ciompi F, Ghafoorian M, van der Laak JAWM, van Ginneken B, Sánchez CI. A survey on deep learning in medical image analysis. *Med Image Anal*. 2017 Dec;42:60-88. doi: 10.1016/j.media.2017.07.005. Epub 2017 Jul 26. PMID: 28778026.
- [8] <https://readmymri.com/blog/how-many-medical-imaging-scans-are-done-per-year> accesat la data 25/05/2026
- [9] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," *arXiv preprint arXiv:1505.04597*, 2015.
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun (2015). Deep Residual Learning for Image Recognition. *CoRR*, abs/1512.03385.
- [11] Baumgartner CF, Kamnitsas K, Matthew J, Fletcher TP, Smith S, Koch LM, Kainz B, Rueckert D. SonoNet: Real-Time Detection and Localisation of Fetal Standard Scan Planes in Freehand Ultrasound. *IEEE Trans Med Imaging*. 2017 Nov;36(11):2204-2215. doi: 10.1109/TMI.2017.2712367. Epub 2017 Jul 11. PMID: 28708546; PMCID: PMC6051487.
- [12] Burgos-Artizzu XP, Coronado-Gutiérrez D, Valenzuela-Alcaraz B, Bonet-Carne E, Eixarch E, Crispí F, Gratacós E. Evaluation of deep convolutional neural networks for automatic classification of common maternal fetal ultrasound planes. *Sci Rep*. 2020 Jun 23;10(1):10200. doi: 10.1038/s41598-020-67076-5. Erratum in: *Sci Rep*. 2022 Jan 31;12(1):1950. doi: 10.1038/s41598-022-06173-z. PMID: 32576905; PMCID: PMC7311420.
- [13] Liu, L., Tang, D., Li, X. et al. Automatic fetal ultrasound image segmentation of first trimester for measuring biometric parameters based on deep learning. *Multimed Tools Appl* **83**, 27283-27304 (2024). <https://doi.org/10.1007/s11042-023-16565-6>
- [14] Krizhevsky, A., Sutskever, I., & Hinton, G. (2012). ImageNet Classification with Deep Convolutional Neural Networks. In *Advances in Neural Information Processing Systems*. Curran Associates, Inc.

- [15] Tajbakhsh N, Shin JY, Gurudu SR, Hurst RT, Kendall CB, Gotway MB, Jianming Liang. Convolutional Neural Networks for Medical Image Analysis: Full Training or Fine Tuning? *IEEE Trans Med Imaging*. 2016 May;35(5):1299-1312. doi: 10.1109/TMI.2016.2535302. Epub 2016 Mar 7. PMID: 26978662.
- [16] Fausto Milletari, Nassir Navab, & Seyed-Ahmad Ahmadi (2016). V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation. *CoRR*, abs/1606.04797.
- [17] Seyed Sadegh Mohseni Salehi, Deniz Erdogmus, & Ali Gholipour (2017). Tversky loss function for image segmentation using 3D fully convolutional deep networks. *CoRR*, abs/1706.05721.
- [18] <https://www.superdatascience.com/blogs/convolutional-neural-networks-cnn-step-4-full-connection> accesat la data 28/05/2026
- [19] <https://learnopencv.com/understanding-convolutional-neural-networks-cnn/> accesat la data de 28/05/2026
- [20] Lefkivits, S., & Laszlo, L. (2022). U-Net architecture variants for brain tumor segmentation of histogram corrected images. *Acta Universitatis Sapientiae, Informatica*, 14, 49-74.
- [21] <https://www.geeksforgeeks.org/machine-learning/u-net-architecture-explained/> accesat la data 29/05/2026
- [22] Taha AA, Hanbury A. Metrics for evaluating 3D medical image segmentation: analysis, selection, and tool. *BMC Med Imaging*. 2015 Aug 12;15:29. doi: 10.1186/s12880-015-0068-x. PMID: 26263899; PMCID: PMC4533825.
- [23] Da Correggio, Karine Souza; Noya Galluzzo, Roberto; Santos, Luís Otávio; Soares Muylaert Barroso, Felipe; Zimmermann Loureiro Chaves, Thiago; Sherlley Casimiro Onofre, Alexandre; von Wangenheim, Aldo (2023), "Fetal Abdominal Structures Segmentation Dataset Using Ultrasonic Images", *Mendeley Data*, VI, doi: 10.17632/4gcpm9dsc3.1
- [24] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [25] Buslaev, A., Iglovikov, V., Khvedchenya, E., Parinov, A., Druzhinin, M., & Kalinin, A. (2020). Albumentations: Fast and Flexible Image Augmentations. *Information*, 11(2), 125.
- [26] Hughes, C. (2022). Transfer Learning on Greyscale Images: How to Fine-Tune Pretrained Models on Black-and-White Datasets. *Towards Data Science*. Disponibil la: <https://towardsdatascience.com/transfer-learning-on-greyscale-images-how-to-fine-tune-pretrained-models-on-black-and-white-9a5150755c7a/> accesat la data de 04/06/2026
- [27] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory F. Diamos, Erich Elsen, David García, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, & Hao Wu (2017). Mixed Precision Training. *CoRR*, abs/1710.03740.
- [28] Razvan Pascanu, Tomás Mikolov, & Yoshua Bengio (2012). Understanding the exploding gradient problem. *CoRR*, abs/1211.5063.

- [29] Karel Zuiderveld. 1994. *Contrast limited adaptive histogram equalization*. *Graphics gems IV*. Academic Press Professional, Inc., USA, 474-485.
- [30] Vijayaraghavan SB. *Ultrasound of the Fetal Gastrointestinal Tract*. *Glob Libr Women's Med*. 2024. ISSN: 1756-2228; DOI: 10.3843/GLOWM.420463
- [31] Nabila Abraham, & Naimul Mefraz Khan (2018). *A Novel Focal Tversky loss function with improved Attention U-Net for lesion segmentation*. *CoRR*, abs/1810.07842.

Anexa 1

Codul folosit pentru antrenarea modelului în Google Collab, în format .ipynb

```
import os
import glob
import re
import numpy as np
from tqdm import tqdm
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from torch.amp import autocast, GradScaler
from sklearn.model_selection import train_test_split
import torchvision.models as models
import albumentations as A
from albumentations.pytorch import ToTensorV2

class Config:
    DATA_DIR = '/content/dataset_local'

    IN_CHANNELS = 1
    BATCH_SIZE = 8
    NUM_EPOCHS = 150
    LEARNING_RATE = 3e-4
    WEIGHT_DECAY = 1e-5
    MAX_GRAD_NORM = 1.0
    PATIENCE_EARLY_STOPPING = 150

    SEED = 42
    DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
    NUM_WORKERS = 2
    PIN_MEMORY = True

# Arhitectura modelului

class DecoderBlock(nn.Module):
    def __init__(self, in_channels, skip_channels, out_channels):
        super().__init__()
        self.up = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)
        self.conv = nn.Sequential(
            nn.Conv2d(in_channels + skip_channels, out_channels, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_channels), nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_channels), nn.ReLU(inplace=True)
        )
```

```

def forward(self, x, skip=None):
    x = self.up(x)
    if skip is not None:
        if x.shape[2:] != skip.shape[2:]:
            x = F.interpolate(x, size=skip.shape[2:], mode='bilinear', align_corners=True)
        x = torch.cat([x, skip], dim=1)
    return self.conv(x)

class MultiOutputUNet(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        backbone = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)
        rgb_w = backbone.conv1.weight.clone()
        backbone.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False)
        backbone.conv1.weight.data = torch.sum(rgb_w, dim=1, keepdim=True)
        self.enc1 = nn.Sequential(backbone.conv1, backbone.bn1, backbone.relu)
        self.pool = backbone.maxpool
        self.enc2 = backbone.layer1
        self.enc3 = backbone.layer2
        self.enc4 = backbone.layer3
        self.enc5 = backbone.layer4
        self.dec4 = DecoderBlock(2048, 1024, 512)
        self.dec3 = DecoderBlock(512, 512, 256)
        self.dec2 = DecoderBlock(256, 256, 128)
        self.dec1 = DecoderBlock(128, 64, 64)
        self.final_conv = nn.Sequential(
            nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True),
            nn.Conv2d(64, 32, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, num_classes, kernel_size=1)
        )

    def forward(self, x):
        a1 = self.enc1(x)
        a2 = self.enc2(self.pool(a1))
        a3 = self.enc3(a2)
        a4 = self.enc4(a3)
        a5 = self.enc5(a4)
        b4 = self.dec4(a5, a4)
        b3 = self.dec3(b4, a3)
        b2 = self.dec2(b3, a2)
        b1 = self.dec1(b2, a1)
        return self.final_conv(b1)

# Funcția de pierdere și metricile

class MultiOrganLoss(nn.Module):

```

```

def __init__(self, num_classes):
    super().__init__()
    pos_weights = torch.tensor([150.0, 15.0, 20.0, 30.0]).view(1, -1, 1, 1).to(Config.DEVICE)
    self.bce = nn.BCEWithLogitsLoss(pos_weight=pos_weights)
    self.alpha = 0.4
    self.beta = 0.6
    self.gamma = 0.75

def forward(self, logits, targets):
    loss_bce = self.bce(logits, targets)
    probs = torch.sigmoid(logits)
    smooth = 1e-5
    tp = (probs * targets).sum(dim=(2, 3))
    fp = ((1 - targets) * probs).sum(dim=(2, 3))
    fn = (targets * (1 - probs)).sum(dim=(2, 3))
    tversky = (tp + smooth) / (tp + self.alpha * fn + self.beta * fp + smooth)
    tversky_loss = torch.pow((1 - tversky), self.gamma).mean()
    return 0.4 * loss_bce + 0.6 * tversky_loss

def compute_metrics(logits, targets, organ_names):
    preds = (torch.sigmoid(logits) > 0.5).float()
    metrics = {organ: {} for organ in organ_names}

    tp = (preds * targets).sum(dim=(0, 2, 3))
    fp = (preds * (1 - targets)).sum(dim=(0, 2, 3))
    fn = ((1 - preds) * targets).sum(dim=(0, 2, 3))
    tn = ((1 - preds) * (1 - targets)).sum(dim=(0, 2, 3))

    smooth = 1e-7

    dsc = (2 * tp) / (2 * tp + fp + fn + smooth)
    jac = tp / (tp + fp + fn + smooth)
    sens = tp / (tp + fn + smooth)
    spec = tn / (tn + fp + smooth)
    prec = tp / (tp + fp + smooth)

    for i, organ in enumerate(organ_names):
        metrics[organ]['dice'] = dsc[i].item()
        metrics[organ]['iou'] = jac[i].item()
        metrics[organ]['sens'] = sens[i].item()
        metrics[organ]['spec'] = spec[i].item()
        metrics[organ]['prec'] = prec[i].item()

    return metrics

# Setul de date

def analyze_and_split_dataset(data_dir, val_ratio=0.15, test_ratio=0.15, seed=Config.SEED):

```

```

all_files = sorted(glob.glob(os.path.join(data_dir, '*.numpy')))
if not all_files:
    raise FileNotFoundError(f"No .numpy files found in {data_dir}")
sample = np.load(all_files[0], allow_pickle=True).item()
organ_names = sorted(sample['structures'].keys())

pat_ids = sorted(set(re.search(r'(P\d+)_IMG', os.path.basename(f)).group(1) for f in all_files))
tr_pats, tmp_pats = train_test_split(pat_ids, test_size=val_ratio + test_ratio, random_state=seed)
vl_pats, ts_pats = train_test_split(tmp_pats, test_size=test_ratio / (val_ratio + test_ratio),
random_state=seed)

def filter_by_patient(p_set):
    return [f for f in all_files if re.search(r'(P\d+)_IMG', os.path.basename(f)).group(1) in
p_set]

train_paths = filter_by_patient(set(tr_pats))
val_paths = filter_by_patient(set(vl_pats))
test_paths = filter_by_patient(set(ts_pats))

print(f"Organs: {organ_names}")
print(f"Train: {len(train_paths)} | Val: {len(val_paths)} | Test: {len(test_paths)}")
return train_paths, val_paths, test_paths, organ_names

class MultiOrganDataset(Dataset):
    def __init__(self, file_paths, organ_names, transform=None):
        self.paths = file_paths
        self.organ_names = organ_names
        self.transform = transform

    def __len__(self):
        return len(self.paths)

    def __getitem__(self, idx):
        data = np.load(self.paths[idx], allow_pickle=True).item()

        image = np.ascontiguousarray(data['image'].astype(np.float32))
        mask_list = [np.ascontiguousarray(data['structures'][organ].astype(np.float32)) for organ in
self.organ_names]

        if self.transform:
            augmented = self.transform(image=image, masks=mask_list)
            image = augmented['image']
            mask_list = augmented['masks']

            masks = torch.stack(
                [m if isinstance(m, torch.Tensor) else torch.from_numpy(np.ascontiguousarray(m)) for
m in mask_list]
            ).float()
        else:
            masks = torch.stack([torch.from_numpy(m) for m in mask_list]).float()

```

```

        if not isinstance(image, torch.Tensor):
            image = torch.from_numpy(np.ascontiguousarray(image))

        if image.ndim == 2:
            image = image.unsqueeze(0)
        elif image.ndim == 3 and image.shape[-1] == 1:
            image = image.permute(2, 0, 1)

        return {'image': image.float(), 'masks': masks}

# Augmentări și pregătirea antrenării

def get_transforms():
    train_transform = A.Compose([
        A.Resize(256, 256),
        A.HorizontalFlip(p=0.5),
        A.RandomRotate90(p=0.5),
        A.RandomBrightnessContrast(brightness_limit=0.2, contrast_limit=0.2, p=0.5),
        A.GaussNoise(p=0.2),
        A.ElasticTransform(alpha=1.0, sigma=50.0, p=0.2),
        A.Normalize(mean=(0.5,), std=(0.5,), max_pixel_value=255.0),
        ToTensorV2()
    ], is_check_shapes=False)

    val_transform = A.Compose([
        A.Resize(256, 256),
        A.Normalize(mean=(0.5,), std=(0.5,), max_pixel_value=255.0),
        ToTensorV2()
    ], is_check_shapes=False)

    return train_transform, val_transform

train_paths, val_paths, test_paths, organ_names = analyze_and_split_dataset(Config.DATA_DIR)
num_classes = len(organ_names)

train_transform, val_transform = get_transforms()

train_ds = MultiOrganDataset(train_paths, organ_names, train_transform)
val_ds = MultiOrganDataset(val_paths, organ_names, val_transform)

train_loader = DataLoader(train_ds, batch_size=Config.BATCH_SIZE, shuffle=True,
                           num_workers=Config.NUM_WORKERS, pin_memory=Config.PIN_MEMORY)
val_loader = DataLoader(val_ds, batch_size=Config.BATCH_SIZE, shuffle=False,
                        num_workers=Config.NUM_WORKERS, pin_memory=Config.PIN_MEMORY)

model = MultiOutputUNet(num_classes).to(Config.DEVICE)
criterion = MultiOrganLoss(num_classes)

```

```

optimizer          = torch.optim.Adam(model.parameters(),          lr=Config.LEARNING_RATE,
weight_decay=Config.WEIGHT_DECAY)

scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=Config.NUM_EPOCHS)

scaler      = GradScaler('cuda')

best_dice   = 0.0
no_improve  = 0

# Bucla de antrenare

for epoch in range(1, Config.NUM_EPOCHS + 1):

    model.train()
    train_loss = 0.0
    bar_train = tqdm(train_loader, desc=f"Epoch {epoch}/{Config.NUM_EPOCHS} [Train]", leave=False)

    for batch in bar_train:
        images, masks = batch['image'].to(Config.DEVICE), batch['masks'].to(Config.DEVICE)

        optimizer.zero_grad()
        with autocast(Config.DEVICE):
            logits = model(images)
            loss = criterion(logits, masks)

        scaler.scale(loss).backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), Config.MAX_GRAD_NORM)
        scaler.step(optimizer)
        scaler.update()

        train_loss += loss.item()
        bar_train.set_postfix_str(f"Loss: {loss.item():.4f}")

    train_loss /= len(train_loader)

    model.eval()
    val_loss = 0.0
    val_stats = {organ: {'dice': [], 'iou': [], 'sens': [], 'spec': [], 'prec': []} for organ in organ_names}
    bar_val = tqdm(val_loader, desc=f"Epoch {epoch}/{Config.NUM_EPOCHS} [Val  ]", leave=False)

    with torch.no_grad():
        for batch in bar_val:
            images, masks = batch['image'].to(Config.DEVICE), batch['masks'].to(Config.DEVICE)

            with autocast(Config.DEVICE):
                logits = model(images)
                loss = criterion(logits, masks)

            val_loss += loss.item()

```



```

        metrics = compute_metrics(logits, masks, organ_names)

        for organ in organ_names:
            for k in val_stats[organ]:
                val_stats[organ][k].append(metrics[organ][k])

val_loss /= len(val_loader)

dice_per_organ = []
print(f"\n{'-'*95}")
print(f"EPOCH {epoch} | Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f}")
print(f"{'-'*95}")
print(f"{'Organ':<15} | {'Dice':<8} | {'IoU':<8} | {'Sens':<8} | {'Spec':<8} | {'Prec':<8}")
print(f"{'-'*95}")

for organ in organ_names:
    d = np.mean(val_stats[organ]['dice'])
    i = np.mean(val_stats[organ]['iou'])
    sn = np.mean(val_stats[organ]['sens'])
    sp = np.mean(val_stats[organ]['spec'])
    pr = np.mean(val_stats[organ]['prec'])
    dice_per_organ.append(d)
    print(f"{organ:<15} | {d:.4f} | {i:.4f} | {sn:.4f} | {sp:.4f} | {pr:.4f}")

avg_val_dice = np.mean(dice_per_organ)
print(f"{'-'*95}")
print(f"MACRO AVG | {avg_val_dice:.4f}")
print(f"{'-'*95}\n")

scheduler.step()

if avg_val_dice > best_dice:
    best_dice = avg_val_dice
    no_improve = 0
    torch.save({'model_state_dict': model.state_dict()}, 'best_model.pt')
    print("New best model saved.")
else:
    no_improve += 1

if no_improve >= Config.PATIENCE_EARLY_STOPPING:
    print("Early stopping triggered.")
    break

print("\nTraining complete.")

```


Anexa 2

Codul aplicației de inferență – demo

```
import cv2
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision.models as models
import albumentations as A
from albumentations.pytorch import ToTensorV2

CLASE = ['artery', 'liver', 'stomach', 'vein']
NR_CLASE = 4

DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'

# Transformare de inferenta: doar normalizare, fara augmentari (identic cu val_transform din antrenare)
aug_val = A.Compose([
    A.Normalize(mean=(0.5,), std=(0.5,), max_pixel_value=255.0),
    ToTensorV2()
])

# Arhitectura modelului - versiunea de inferenta (identica cu MultiOutputUNet din antrenare,
weights=None)

class UpBlock(nn.Module):
    def __init__(self, ch_in, ch_skip, ch_out):
        super().__init__()
        self.upsample = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)
        total_in = ch_in + ch_skip
        self.convblock = nn.Sequential(
            nn.Conv2d(total_in, ch_out, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(ch_out),
            nn.ReLU(inplace=True),
            nn.Conv2d(ch_out, ch_out, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(ch_out),
            nn.ReLU(inplace=True)
        )

    def forward(self, x, skip=None):
        x = self.upsample(x)
        if skip is not None:
            if x.shape[2:] != skip.shape[2:]:
                x = F.interpolate(x, size=skip.shape[2:], mode='bilinear', align_corners=True)
            x = torch.cat([x, skip], dim=1)
        return self.convblock(x)
```

```

class SegModel(nn.Module):
    def __init__(self, nr_clase):
        super().__init__()
        backbone = models.resnet50(weights=None)
        backbone.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False)

        self.s1 = nn.Sequential(backbone.conv1, backbone.bn1, backbone.relu)
        self.pool = backbone.maxpool
        self.s2 = backbone.layer1
        self.s3 = backbone.layer2
        self.s4 = backbone.layer3
        self.s5 = backbone.layer4

        self.up4 = UpBlock(2048, 1024, 512)
        self.up3 = UpBlock(512, 512, 256)
        self.up2 = UpBlock(256, 256, 128)
        self.up1 = UpBlock(128, 64, 64)

        self.cap = nn.Sequential(
            nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True),
            nn.Conv2d(64, 32, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, nr_clase, kernel_size=1)
        )

    def forward(self, x):
        f1 = self.s1(x)
        f2 = self.s2(self.pool(f1))
        f3 = self.s3(f2)
        f4 = self.s4(f3)
        f5 = self.s5(f4)
        d4 = self.up4(f5, f4)
        d3 = self.up3(d4, f3)
        d2 = self.up2(d3, f2)
        d1 = self.up1(d2, f1)
        return self.cap(d1)

# Pipeline de preprocesare

def norm_to_uint8(img):
    if img.dtype == np.uint8:
        return img
    tmp = img.astype(np.float32)
    tmp = tmp - tmp.min()
    mx = tmp.max()
    if mx > 0:
        tmp = tmp / mx * 255.0

```

```

    return np.clip(tmp, 0, 255).astype(np.uint8)

def filtru_meniu(img, raza=3):
    img = img.astype(np.float32)
    bordura = np.pad(img, raza, mode="reflect")
    H, W = img.shape
    dim = 2 * raza + 1

    # imagine integrala pentru media locala eficienta
    integ = np.zeros((bordura.shape[0] + 1, bordura.shape[1] + 1), dtype=np.float32)
    integ[1:, 1:] = bordura.cumsum(axis=0).cumsum(axis=1)

    a = integ[dim : dim+H, dim : dim+W]
    b = integ[:H, dim : dim+W]
    c = integ[dim : dim+H, :W]
    d = integ[:H, :W]
    return (a - b - c + d) / (dim * dim)

def dilate_mask(mask, raza=1):
    mask = mask.astype(bool)
    tmp = np.pad(mask, raza, mode="constant", constant_values=False)
    rezultat = np.zeros_like(mask)
    for dy in range(2*raza + 1):
        for dx in range(2*raza + 1):
            rezultat |= tmp[dy : dy+mask.shape[0], dx : dx+mask.shape[1]]
    return rezultat

def erode_mask(mask, raza=1):
    mask = mask.astype(bool)
    tmp = np.pad(mask, raza, mode="constant", constant_values=False)
    rezultat = np.ones_like(mask)
    for dy in range(2*raza + 1):
        for dx in range(2*raza + 1):
            rezultat &= tmp[dy : dy+mask.shape[0], dx : dx+mask.shape[1]]
    return rezultat

def curata_zgomot(mask, raza=1):
    return dilate_mask(erode_mask(mask, raza), raza)

def elimina_suprapunere_text(img):
    img = norm_to_uint8(img)
    medie_loc = filtru_meniu(img, raza=10)
    masca_text = (img >= medie_loc + 22) | (img <= medie_loc - 22)
    masca_text = curata_zgomot(masca_text, raza=2)
    masca_text = dilate_mask(masca_text, raza=3)
    fundal = filtru_meniu(img, raza=12).astype(np.uint8)
    curat = img.copy()
    curat[masca_text] = fundal[masca_text]

```

```

    return curat

def preprocesare(img):
    curat = elimina_suprapunere_text(img)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    return clahe.apply(curat)

# Citirea datelor

def citeste_imagine_npy(cale):
    date = np.load(cale, allow_pickle=True)
    if isinstance(date, np.ndarray) and date.shape == () and date.dtype == object:
        date = date.item()
    if isinstance(date, dict):
        if 'image' not in date:
            raise ValueError("Dict-ul .npy nu contine cheia 'image'.")
        img = date['image']
    else:
        img = date
    img = np.asarray(img).astype(np.float32)
    if img.ndim == 3 and img.shape[-1] == 3:
        img = img[:, :, 0]
    if img.ndim == 3:
        img = np.squeeze(img)
    if img.ndim != 2:
        raise ValueError(f"Imaginea trebuie sa fie 2D dupa procesare, am primit shape {img.shape}.")
    return img

# Incarcarea modelului

def incarca_model(model_path, device):
    net = SegModel(NR_CLASE).to(device)
    ckpt = torch.load(model_path, map_location=device)
    if isinstance(ckpt, dict) and 'model_state_dict' in ckpt:
        ckpt = ckpt['model_state_dict']
    net.load_state_dict(ckpt)
    net.eval()
    return net

# Inferenta

def inferenta(model, img2d, device):
    aug = aug_val(image=img2d)
    t = aug['image'].unsqueeze(0).to(device)
    with torch.no_grad():
        logits = model(t)
        pred = (torch.sigmoid(logits) > 0.5).float()
    return pred[0].cpu().numpy()

```

